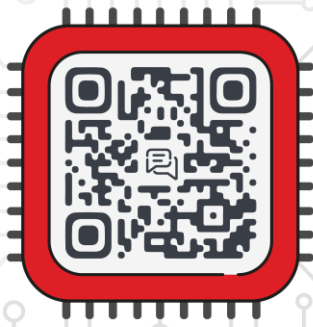




6



62



9300 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

تقنيات تدريب الشبكات العصبية

عملي مشترك



RB Informatcs; 4/12/2025

محتوى مجاني غير مخصص للبيع التجاري

الشبكات العصبونية

مقدمة:

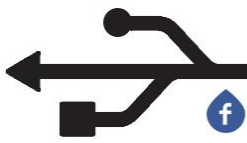
- تدريب الشبكات العصبية بكفاءة هو تحدي أساسي يتطلب أكثر من مجرد خوارزمية بسيطة، بل هو يجمع بين اختيار الاستراتيجيات الصحيحة لضمان أفضل أداء.
- في هذا الالاب، سنتعمق في صميم عملية التدريب، حيث سنتعرف على تقنيات تحسين متقدمة تبحث عن المعاملات المثالية للأوزان بشكل أذكى وأسرع، سندرك أن نقطة البداية، أي القيم الأولية لهذه الأوزان، لها تأثير حاسم على نجاح العملية .
- ولأن النموذج قد يقع في فخ الإفراط في التعلم (overfitting) ويفقد قدرته على التعميم، سنستكشف طرق تنظيم فعّالة مثل dropout و weight decay لمنع هذه المشكلة.
- مساحة البحث عن الحل الأمثل معقدة جداً، خاصة في الشبكات العميقة التي تحتوي على ملايين الأوزان لهذا السبب نحتاج إلى تقنيات ذكية للبحث عن هذه القيم المثلى.
- لذلك، سنضيف إلى أدواتنا تقنية حديثة وقوية هي تطبيع الدفعات (batch normalization) التي أدت دور فعّال في سرعة واستقرار تدريب الشبكات العميقة .

Updating Parameters تحديث المعاملات (رحلة البحث عن الحل الأمثل):

1. الهدف الأساسي: إيجاد الحل الأمثل

؟ ما هو الهدف الأساسي من تدريب الشبكة العصبية؟

- الهدف هو العثور على أفضل المعاملات (Parameters) التي تجعل قيمة دالة الخسارة (Loss Function) أقل ما يمكن (مو شرط أقل قيمة ممكن تكون أعلى قيمة وذلك حسب المسألة الموجودة و لكن نحن هدفنا دالة الخسارة لذلك نريد أقل قيمة)
- عملية البحث عن هذه المعاملات المثالية تسمى التحسين (Optimization)



2. المشكلة: رحلة البحث المعقدة

سنتحدث عن قصة مغامر ونربطها مباشرة بمفهوم تحسين الشبكات العصبونية لفهم الفكرة :
لكي نفهم التحدي الذي نواجهه في التحسين (Optimization) ، سنتحدث عن قصة المغامر :

شخصيات ومكان القصة

- المغامر (The Adventurer) :
- هو شخص ما أو بالأحرى، هو عملية تدريب الشبكة العصبونية نفسها مهمته هي البحث عن الأفضل
- المنطقة الجافة الشاسعة والوادي العميق (The Vast Dry Region & Deep Valley) :
- هذه المنطقة هي مساحة المعاملات (Parameter Space) تخيل عالماً افتراضياً ضخماً جداً، كل نقطة فيه تمثل مجموعة مختلفة من أوزان الشبكة العصبونية
- بعض النقاط تكون سيئة (خسارة عالية)، وبعضها جيدة (خسارة منخفضة)
- هدفنا هو العثور على المكان العميق (The Deep Place) أي الوصول لقاع أدنى وادي، وهو نقطة الحل الأمثل التي تكون فيها دالة الخسارة بأقل قيمة ممكنة
- القيود الصارمة (The Strict Restrictions) :
- المغامر وضع على نفسه قيدين:

1. لا يوجد خريطة (No Map) :

هذا يعني أننا لا نعرف مكان الحل الأمثل مسبقاً (مكان قاع أدنى وادي لا نعرف احداثياته)
لا يوجد دليل يقول لنا اذهب 100 خطوة إلى الشمال ثم 50 خطوة شرقاً أي لا نملك GPS

2. مغطى العينين (Blindfolded) :

هذا يعني أننا لا نستطيع رؤية مساحة المعاملات بأكملها من الأعلى
لا يمكننا رؤية شكل الوادي بالكامل لتتخذ الطريق الأسرع

الأداة الوحيدة المتاحة

ميلان الأرض تحت قدميه (The Inclination of the Ground) أي سنستعين بالميل حتى أستطيع الوصول لأدنى قاع:

- على الرغم من أنه معصوب العينين، إلا أن المغامر يمكنه أن يشعر بميلان الأرض تحت قدميه في المكان الذي يقف فيه فقط
- هذا الإحساس بالميلان هو التدرج (Gradient) في عالم الشبكات العصبونية
- هذه المعلومة الوحيدة التي نملكها
- في أي لحظة: نخبرنا بالاتجاه الذي تصعد فيه قيمة الخسارة بشكل أسرع من نقطتنا الحالية

استراتيجية المغامر البسيطة (وهي استراتيجية SGD)

ماذا سيفعل المغامر؟

- فكرته بسيطة ومنطقية: سيتحرك في الاتجاه الذي يكون فيه الميلان للأسفل هو الأشد أي المكان الأكثر انحداراً سيخطو باتجاهه

- إذا شعر أن الأرض تميل بشدة نحو اليسار، سيتحرك خطوة إلى اليسار
- سيعيد هذه العملية مراراً وتكراراً، على أمل أن يوصله هذا المسار المتعرج يوماً ما إلى قاع الوادي العميق

ماذا يعني هذا في عالم الشبكات؟

- هذه الاستراتيجية هي بالضبط تعريف الانحدار التدريجي العشوائي (SGD) الخوارزمية تحسب التدرج (ميلان الأرض)، ثم تقوم بتحديث الأوزان (تتحرك خطوة) في الاتجاه المعاكس للتدرج (لأن التدرج يشير للأعلى، ونحن نريد النزول للأسفل)

ملاحظة :

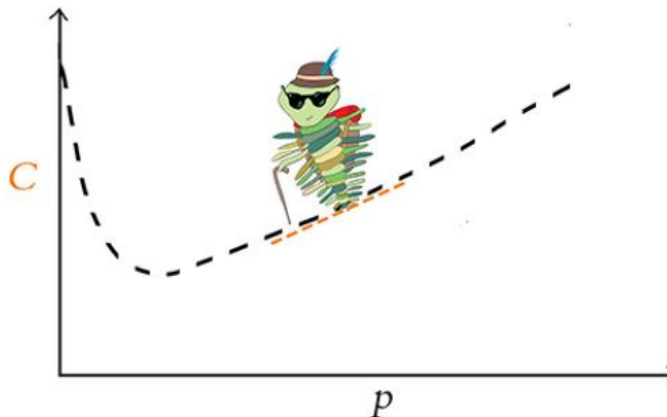
- سابقاً كنا نحدّث القيم بشكل تكراري من خلال خوارزمية gradient descent التي تحدثنا عنها وكنا نطلق عليها اسم Full Batch أي تم حساب دالة الخسارة على الداتا كلها أي عندما نحسب ل gradient يتم حسابه نتيجة كل الداتا
- فعندما يكون حجم الباتش = 1 هذا يدعى stochastic gradient descent
- وعندما يكون حجم الباتش: $1 < \text{batch size} < N$ (N هو حجم الداتا كاملة) هذا يدعى: mini-batch Gradient Descent
- لماذا تسمى stochastic؟؟ لأن عملية اختيار الباتش تكون عملية عشوائية

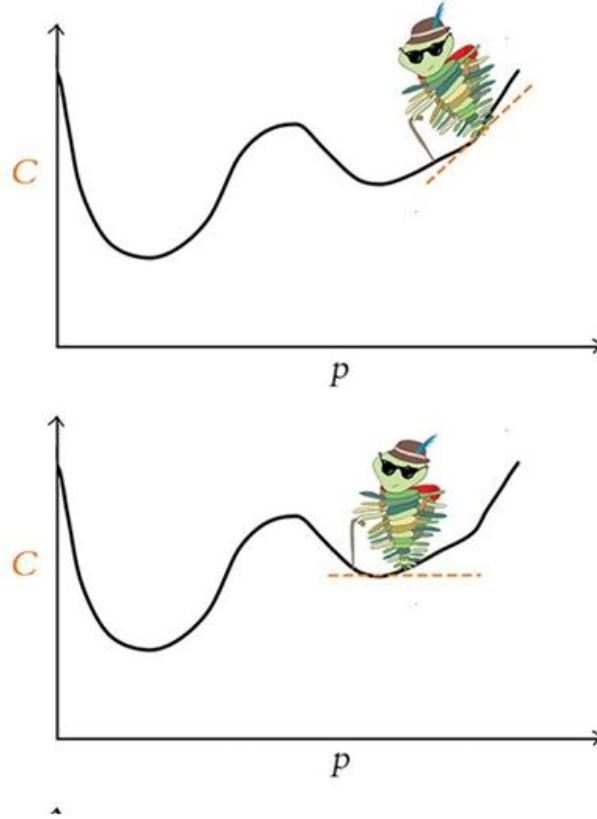
الخلاصة والرسالة من القصة

1. طبيعة المشكلة:
 2. البحث عن الحل الأمثل في الشبكات العصبونية هو رحلة في عالم مظلم وغير معروف، تماماً مثل المغامر الأعمى
 3. استراتيجية SGD : هي استراتيجية بسيطة تعتمد على المعلومة المحلية المتاحة فقط (التدرج)، وهي بالفعل أذكى من البحث العشوائي
 3. المشكلة المحتملة :
- لكن هل هذه الاستراتيجية هي الأفضل دائماً؟
- في حال كان الوادي طويل وضيق (مثل طبق) فالمغامر الذي يتبع الميلان سيتحرك بشكل متعرج (Zigzag) من جدار إلى آخر، وهذا سيكون طريقاً بطيئاً وغير فعال للوصول إلى القاع

وهنا يظهر في ذهننا سؤال هل هناك طرق أذكى من مجرد اتباع الميلان؟

هنا يأتي دور تقنيات تحسين مثل Momentum و AdaGrad و Adam ، التي تحاول أن تكون أكثر ذكاءً من مغامرنا الأعمى للوصول إلى المكان العميق بشكل أسرع وأكثر كفاءة





الانحدار التدريجي العشوائي SGD

بعد أن فهم مغامرنا صعوبة المهمة وهي البحث عن المكان العميق في عالم مظلم وبدون خارطة في يده وحاجته إلى استراتيجية بسيطة ليباشر رحلته
وهنا يأتي دور SGD أو الانحدار التدريجي العشوائي، وهي أول وأبسط بوصلة يمكن أن يستخدمها حيث تحدثنا عن استراتيجية هذه البوصلة

المعادلة:

$$w \leftarrow w - \eta \frac{\partial l}{\partial w}$$

الموقع الجديد ← الموقع القديم - (حجم الخطوة × شدة الانحدار)

لنشرح هذه المعادلة :

▪ w الموقع القديم :

هو مكان وقوف المغامر حالياً (أو قيم أوزان شبكتنا العصبية)

▪ $\partial L / \partial w$ شدة الانحدار :

هو التدرج الذي يشعر به المغامر، إنه يشير إلى اتجاه الصعود الأسرع، لذا سنستخدم الإشارة السالبة لنقله إلى اتجاه النزول.

▪ η حجم الخطوة : هذا هو معدل التعلم (Learning Rate) :

هل سيخطو المغامر خطوات حذرة وصغيرة (مثل 0.001) أم خطوات جريئة وكبيرة (مثل 0.1)؟ هذا هو القرار الذي نتخذه قبل بدء الرحلة هذا يؤثر على سرعة ودقة وصوله للهدف

بناء الأداة: implement SGD as a class

الآن، سنصنع كلاس اسمه SGD

```
3 class SGD:
4     def __init__(self, lr=0.01):
5         self.lr = lr
6     def update(self, params, grads):
7         for key in params.keys():
8             params[key] -= self.lr * grads[key]
```

الشرح :

- : __init__
- عندما نريد إنشاء أداة SGD جديدة، نحدد لها حجم الخطوة (learning rate) الذي سنستخدمه
- : Update
- هذه هي الوظيفة الأساسية
- نعطيهما خريطة بمواقعنا الحالية (params) وتقديرًا بميلان الأرض (grads) ، وهي تقوم بتحديث كل موقع في الخريطة

كيف نستخدم الأداة في رحلتنا؟

الآن، لنرى كيف يستخدم المغامر هذه الأداة في رحلته الفعلية.
الكود التالي يوضح الخطوات التي تحدث في كل دورة من دورات التدريب:

```
network = TwoLayerNet(...)
optimizer = SGD()
for i in range(10000):
    ...
    x_batch, t_batch = get_mini_batch(...) # Mini-batch
    grads = network.gradient(x_batch, t_batch)
    params = network.params
    optimizer.update(params, grads)
    ...
```

- هذا الكود يوضح أن عملية التحديث أصبحت **معزولة ومنظمة** كل ما علينا فعله هو حساب التدرجات وتمريضها للأداة optimizer
- **حيث** optimizer (المُحسِّن) هو مرشد المغامر الذي يتولى مهمة تحديث المواقع, كل ما على المغامر فعله هو تزويده بالمعلومات الصحيحة، والمرشد يتكفل بالباقي

لماذا نفصلها في كلاس؟

- تخيل أن مغامرنا اكتشف أن بوصلة SGD البسيطة تجعله يسير في طريق متعرج وغير فعال هل يعني هذا أنه يجب عليه إعادة بناء خطته بأكملها؟ لا
- الجواب يكمن في المرونة والتنظيم (Modularity) :
- المتغير optimizer وظيفته الوحيدة هي تحديث المعاملات
- الذكاء الحقيقي هنا :
- لقد صممنا جميع أدواتنا المحسنة مثل SGD ، وغيرها التي سنراها لاحقاً لتكون لها نفس الواجهة: دالة `update(params, grads)`
- ماذا يعني هذا عملياً؟
- يعني أنه إذا قررنا أن استراتيجية المغامر (SGD) بطيئة وغير فعالة، وأردنا تجربة استراتيجية أذكى مثل Momentum، فكل ما علينا فعله هو تغيير سطر واحد فقط!

بدلاً من استخدام استراتيجية (SGD) البسيطة :

`optimizer = SGD()`

نستخدم استراتيجية Momentum الأكثر ذكاءً :

`optimizer = Momentum()`

وباقى الكود يبقى كما هو دون أي تغيير! هذا يجعل تجربة خوارزميات تحسين مختلفة أمراً سهلاً ومنظماً للغاية، وهو ما يجعل الكود احترافياً وقابلاً للتطوير

Disadvantages of Stochastic Gradient Descent (SGD)

كما تحدثنا على الرغم من أن SGD بسيطة وسهلة التنفيذ، إلا أنها قد تكون غير فعالة وبطيئة في بعض المشاكل لفهم السبب، سنستخدم مثلاً

أولاً:

المثال: لتكن لدينا دالة الخسارة التالية : $f(x, y) = (1/20)x^2 + y^2$

هذه الدالة بسيطة لكنها تكشف عن عيب كبير في SGD لماذا؟

لأن شكلها ليس متساوياً من جميع الجهات

1. شكل الدالة :

- تخيل أن هذه الدالة تشكل وعاءً أو سطحاً ممدوداً بشكل كبير في اتجاه محور x-axis
- في اتجاه المحور (y-axis)، يكون الوعاء حاداً وعميقاً
- في اتجاه المحور (x-axis)، يكون الوعاء عريضاً ومسطحاً جداً

2. خطوط الكونتور (Contour Lines) :

إذا نظرنا إلى خريطة الكونتور لهذه الدالة، سنجد أنها على شكل قطع ناقص (ellipses) ممتدة في اتجاه محور الاكسات

- تذكير:** كل خط كونتور يربط بين النقاط التي لها نفس القيمة للدالة $f(x, y)$ أي كل رقم ب الخط هو قيمة ل z عند x, y (قيمة التابع)
- الخطوط الداخلية تمثل قيماً أقل (أقرب للقاع (للهدف))
- الخطوط الخارجية تمثل قيماً أعلى (أبعد عن القاع (للهدف))
- عندما تكون الخطوط بعيدة عن بعضها تكون gradient صغيرة
- عندما تكون الخطوط قريبة من بعضها تكون gradient كبيرة
- الملاحظة المهمة :

المسافة بين الخطوط في اتجاه المحور (y) صغيرة (انحدار حاد)، بينما المسافة في اتجاه المحور السيني (x) كبيرة (انحدار مسطح)

ثانياً :

عيب SGD الأساسي: المسار المتعرج (Zigzag Path)

؟ الآن، لنجيب على السؤال: لماذا تواجه SGD صعوبة في إيجاد الحل الأمثل لهذه الدالة؟

1. اختلاف حدة التدرجات:

- في اتجاه المحور (x):
- التدرج (gradient) صغير جداً لماذا؟
- لأن المعامل هو 20/1 في هذا المثال هذا يعني أن الميل في هذا الاتجاه ضعيف جداً
- في اتجاه المحور (y):
- التدرج كبير جداً لماذا؟ لأن المعامل هو 1 في هذا المثال وهذا يعني أن الميل في هذا الاتجاه حاد

2. كيف تتحرك SGD ؟

- SGD بسيطة جداً: في كل خطوة، ننظر باتجاه الصعود الأكثر حدة وتتحرك في عكسه
- بسبب الانحدار الحاد في اتجاه y، ستقوم SGD بخطوات كبيرة جداً في هذا الاتجاه
- بسبب الانحدار الضعيف في اتجاه x، ستقوم SGD بخطوات صغيرة جداً في هذا الاتجاه

3. النتيجة: المسار المتعرج

لأن التدرجات في معظم النقاط لا تشير مباشرة إلى نقطة الصغرى (0,0)، فإن SGD لا تسير في خط مستقيم. بدلاً من ذلك، فإنها:

- تقفز قفزة كبيرة في اتجاه y
- ثم تقوم بتصحيح صغير في اتجاه x
- ثم تقفز مرة أخرى في اتجاه y المعاكس
- وهكذا...

هذا يخلق مساراً متعرجاً (zigzag) غير فعال للغاية، حيث تقضي SGD وقتاً طويلاً في التذبذب يميناً ويساراً قبل أن تصل إلى القاع ببطء.

ثالثاً:

مشاكل أخرى يواجهها SGD :

هذه المشكلة ليست محصورة على هذا المثال فقط

SGD تواجه صعوبات في أنواع أخرى من الأسطح:

1. الأسطح غير المحدبة (Non-convex surfaces) :

التي تحتوي على العديد من القيعان المحلية (أشد انحدار محلي local minima) قد يعلق SGD في قاع محلي بدلاً من الوصول إلى القاع العام (global minimum)

2. المناطق المسطحة (Saddle points) :

عندما تكون التدرجات قريبة من الصفر في جميع الاتجاهات، يصبح SGD بطيئاً جداً وكأنه عالق

رسم 1



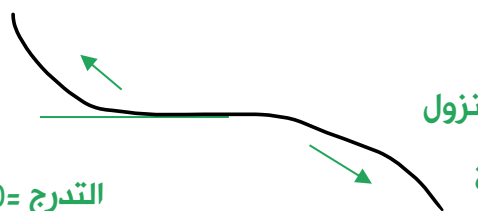
التدرج = 0

3. الانحناء العالي (High curvature) :

عندما يكون السطح شديد الانحدار في اتجاه ومسطح في اتجاه آخر (مثل مثالنا)، يؤدي إلى تذبذب وتقارب بطيء

رسم 2

خسارة كبيرة



التدرج = 0

خسارة صغيرة

لنعود إلى مثال المغامر (SGD) لنرى كيف يتحرك في هذا الوادي:

مغامرنا الأعمى يتبع استراتيجية SGD : اتجه في اتجاه أشد انحدار للأسفل

1. البداية :

المغامر يقف على حافة الوادي في جهة اليسار. هدفه هو الوصول إلى أعمق نقطة في المنتصف

2. الخطوة الأولى:

يشعر بقدميه أن أشد انحدار هو تقريبًا مباشرة للأسفل باتجاه الجدار (المحور y) بناءً على استراتيجيته (SGD) ، يأخذ خطوة كبيرة وقوية في هذا الاتجاه

3. النتيجة:

هذه الخطوة الكبيرة تنقله إلى الجانب الآخر من الوادي (إلى اليمين)، دون أن يتقدم كثيرًا نحو المركز. هو الآن في نفس الارتفاع تقريبًا، لكن على الجانب المقابل

4. الخطوة الثانية :

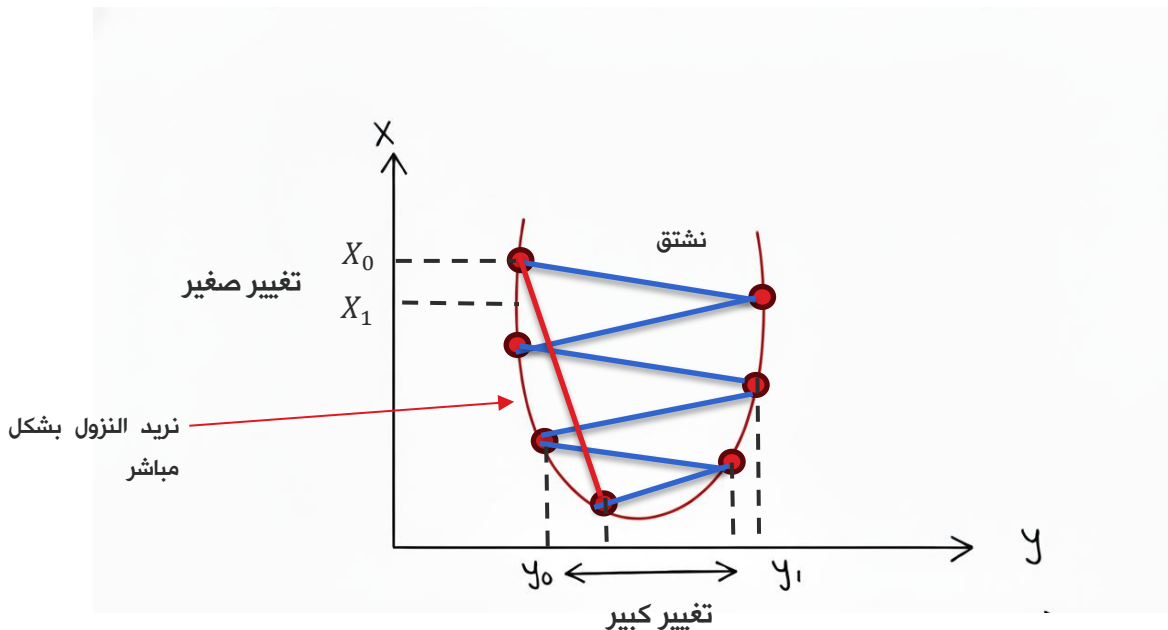
الآن، يشعر مرة أخرى أن أشد انحدار هو مباشرة للأسفل... لكن في الاتجاه المعاكس! فيأخذ خطوة كبيرة أخرى ويعود إلى الجانب الأيسر

5. المسار:

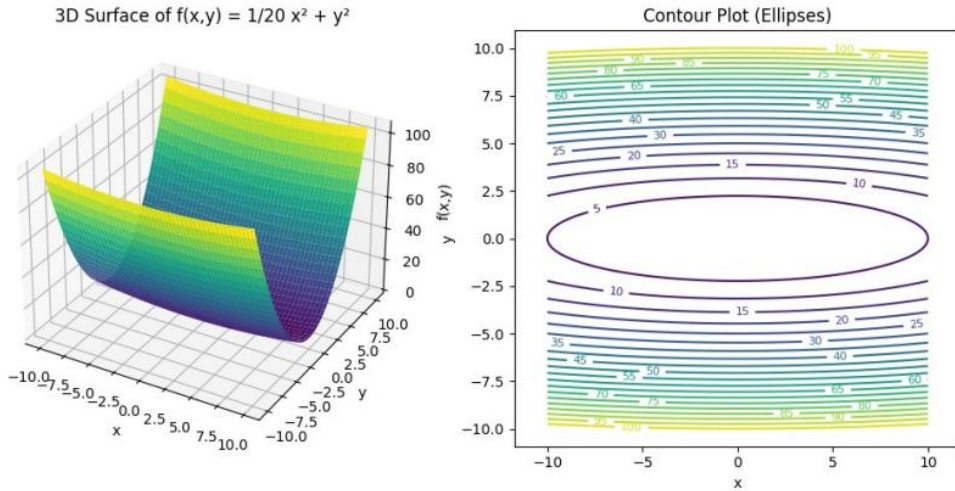
يتكرر هذا الأمر مرارًا وتكرارًا. المغامر يقفز من جانب إلى آخر (حركات كبيرة في اتجاه y) ، وتقدمه البطيء الوحيد نحو المركز (الهدف) يكون عن طريق قفزات صغيرة جدًا لا إرادية في اتجاه x

النتيجة النهائية: مسار متعرج (Zigzag Path)

- يبدو مسار المغامر وكأنه ينصدم بجدران الوادي ذهابًا وإيابًا، بدلاً من التحرك بسلاسة نحو المركز
 - هذا المسار غير فعال للغاية ويستغرق وقتًا طويلاً للوصول إلى الهدف (المكان العميق)
- صورة تعبيرية لحركة المغامر :



الآن لنرى رسمة التابع مع مخطط ل Contour :



نلاحظ ان مجال القيم على x وهو $[-10, 10]$ اكثر من مجال القيم على y وهو $[-2.5, 2.5]$ اي متمدد اكثر على x

في كود رسم التابع يوجد لدينا تعليمة meshgrid لتحدث عن آلية عملها :

- نريد حساب قيمة دالة الخسارة عند كل ثنائية
- مثال : ليكن لدينا فضاء x هو $[1, 2, 3]$ وفضاء y هو ايضاً $[1, 2, 3]$ فما هي الثنائيات الموجودة لدينا؟
- الثنائيات هي 9 عبارة عن $(1,1)$ $(1,2)$ $(1,3)$ $(2,1)$... للوصول إلى $(3,3)$
- فالتابع meshgrid يقوم بتحويل متجهين (أو ثلاثة) إلى مصفوفتين (أو ثلاث) تمثلان جميع الثنائيات الممكنة للإحداثيات

لنعود للكود الموجود لدينا:

الخطوة الأولى: إنشاء المحاور

```
x = np.linspace(-10, 10, 200)
```

```
y = np.linspace(-10, 10, 200)
```

```
: np.linspace(-10, 10, 200)
```

هذه الدالة تنشئ مصفوفة أحادية البعد (1D array) (أو متجه) تحتوي على 200 رقماً، موزعة بالتساوي بين -10 و 10

: x

هذه المصفوفة تمثل المحور (x-axis) قيمها ستكون مثل $[-10, -9.9, -9.8, \dots, 9.9, 10]$

: y

هذه المصفوفة تمثل المحور (y-axis) قيمها ستكون أيضاً $[-10, -9.9, -9.8, \dots, 9.9, 10]$

المشكلة الآن :

لدينا (مصفوفتان D1)، لكننا لا نملك إحداثيات النقاط داخل الشبكة. كيف نعرف إحداثيات النقطة التي تقع عند تقاطع الصف الخامس والعمود العاشر؟

هنا يأتي دور meshgrid

الخطوة الثانية: بناء شبكة الإحداثيات

▪ $X, Y = \text{np.meshgrid}(x, y)$

هذه هي الخطوة الأساسية:

Meshgrid تأخذ المصفوفتين D1 (y و x) وتنتج مصفوفتين ثنائيتين البعد (2D arrays) Y و X

▪ مصفوفة الإحداثيات X :

- هي عبارة عن نسخة من مصفوفة X ولكن مكررة لكل صف في الشبكة
- إذا كان لدينا 200 نقطة في y ، فسيكون لدينا 200 صف
- النتيجة: مصفوفة X بحجم (200, 200). كل صف في هذه المصفوفة هو نسخة طبق الأصل من x الأصلية

▪ مصفوفة الإحداثيات Y :

- هي عبارة عن نسخة من مصفوفة Y ولكن مكررة لكل عمود في الشبكة
- إذا كان لدينا 200 نقطة في x ، فسيكون لدينا 200 عمود
- النتيجة: مصفوفة Y بحجم (200, 200). كل عمود في هذه المصفوفة هو نسخة طبق الأصل من y الأصلية

لماذا هذا مفيد؟

الآن بعد أن أصبح لدينا مصفوفتا X و Y ، يمكننا حساب قيمة الدالة $f(x, y)$ للشبكة بأكملها دون استخدام حلقات for بطيئة

لنرى كيف :

الدالة التي نريد رسمها هي

$$f(x, y) = (1/20) * x^2 + y^2$$

باستخدام meshgrid، يمكننا حسابها ببساطة كالتالي:

$$Z = (1/20) * X ** 2 + Y ** 2$$

ماذا يحدث هنا؟

- يتم استخدام خاصية البث (Broadcasting)
- عندما تكتب $X ** 2$ ، فإنها تربع كل عنصر في مصفوفة X الضخمة
- وعندما تضيف $Y ** 2$ ، فإنها تضيف العنصر المقابل في كل مصفوفة
- النتيجة Z ستكون مصفوفة بحجم (200, 200)
- حيث $Z[i, j] = (1/20) * X[i, j] ** 2 + Y[i, j] ** 2$

- وهذا بالضبط ما نريده $X[i, z]$ هو إحداثي الاكس للنقطة في الصف z والعمود i ، و $Y[i, z]$ هو إحداثي لنفس النقطة

الخلاصة meshgrid :

- هي أداة أساسية في الرسم البياني العلمي (Scientific Plotting) وظيفتها هي تحويل محاور الإحداثيات أحادية البعد إلى شبكات إحداثيات ثنائية البعد
- هذا يسمح لك بحساب وتقييم دالة ثنائية المتغيرات $f(x, y)$ على كامل مساحة الرسم بكفاءة عالية جداً، وهو أمر ضروري لرسم الأسطح ثلاثية الأبعاد والخرائط الكنتورية (contour plots).

الدوال المحدبة (Convex) وغير المحدبة (Non-Convex) :

مثال بسيط لفهم الفكرة :

لنعود لمثال البحث عن أدنى نقطة في منطقة ما:

الحالة المحدبة (Convex)

- المنطقة عبارة عن وعاء أملس وبسيط
- مهما بدأت من أي نقطة على حافة الوعاء، فإنك إذا درجت كرة، فستتوقف حتماً في أدنى نقطة في القاع
- لا يوجد أدنى نقطة أخرى

الحالة غير المحدبة (Non-Convex)

- المنطقة عبارة عن تضاريس معقدة بها العديد من الوديان والتلال
- إذا درجت كرة، فقد تتوقف في وادٍ صغير (حد أدنى محلي local) وليس بالضرورة في أعماق وادٍ في المنطقة بأكملها (الحد الأدنى العام global)

هذا هو جوهر الفرق بين الدوال المحدبة وغير المحدبة في عالم التحسين

لنتحدث الآن بالتفصيل :

أولاً: الدوال المحدبة (Convex Functions)

التعريف والخصائص :

الدالة المحدبة هي دالة لها شكل محدب للأعلى (مثل الوعاء). أهم خصائصها:

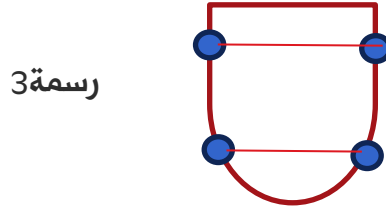
1. حد أدنى واحد فقط (Single Global Minimum) :
2. أي حد أدنى محلي هو حد أدنى عام :
- لا يمكن أن يكون هناك وادٍ صغير منفصل أي وادٍ تجده هو بالضرورة أعماق وادٍ في المنطقة

3. الضمان الرياضي :

إذا تحركت نحو الأسفل باستمرار، فأنت مضمون للوصول إلى الحل الأمثل

مثال بسيط :

الدالة التربيعية البسيطة مثل $f(x) = x^2$ هي دالة محدبة شكلها يشبه حرف U ، ولها نقطة واحدة فقط هي الأدنى عند $x = 0$ إذا رسمت خط مستقيم بهذا الشكل لن يتقاطع مع نقطة ثالثة:



ثانياً: الدوال غير المحدبة (Non-Convex Functions)

التعريف والخصائص :

الدالة غير المحدبة هي أي دالة لا تنطبق عليها خصائص الدوال المحدبة شكلها معقد ومنحني.

1. عديد من الحدود الدنيا المحلية (Multiple Local Minima) :

تحتوي على العديد من الوديان الصغيرة التي قد يعلق فيها المحسن (Optimizer)

2. نقاط السرج (Saddle Points) :

مناطق تكون مسطحة في اتجاه ومنحدرة في اتجاه آخر. يمكن أن تبطل المحسن بشكل كبير لأن التدرجات تكون صغيرة جداً

3. لا يوجد ضمان بالوصول للحل الأمثل :

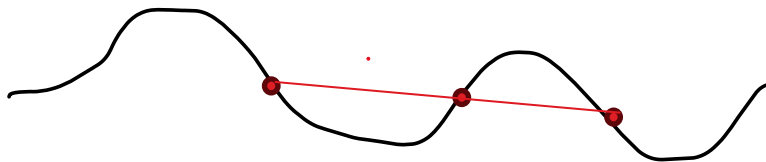
قد يعلق المحسن في حد أدنى محلي سيء ولا يتمكن من الوصول إلى الحد الأدنى العام (أفضل حل)

مثال بسيط:

دالة مثل $f(x) = x^4 - x^2$ لها شكل متعرج، حيث ترى قمة صغيرة بين وديان. الوادي الصغير هو حد أدنى محلي، لكنه ليس الأدنى على الإطلاق.

إذا رسمت مثلاً هذا الشكل ورسمت خط مستقيم رح يتقاطع مع أكثر من نقطة :

رسم 4



1. ثالثاً: جدول مقارنة بسيط :

الميزة (Feature)	الدالة المحدبة (Convex Function)	الدالة غير المحدبة
الشكل	وعاء املس وبسيط	تضاريس معقدة بها تلال ووديان
الحدود الدنيا	حد أدنى واحد فقط (Global Minimum)	العديد من الحدود الدنيا المحلية (Local Minima)
نقاط السرج	لا توجد	شائعة جداً
ضمان التحسين	مضمون الوصول إلى الحل الأمثل	غير مضمون، قد يعلق في حل سيء
صعوبة التحسين	سهلة نسبياً	صعبة ومعقدة للغاية

رابعاً: الربط بالتعلم الآلي (Machine Learning) والتعلم العميق (Deep Learning)

هنا يصبح المفهوم حيويًا لفهم لماذا نستخدم نماذج وتقنيات معينة

في التعلم الآلي التقليدي (Classic Machine Learning)

يتم تصميم دالة الخسارة (Loss Function) لتكون محدبة (Convex)

في التعلم العميق (Deep Learning)

- التعلم العميق هو عالم مختلف تماماً.
 - دوال الخسارة في الشبكات العصبية العميقة تكون دائماً تقريباً غير محدبة (Non-Convex) لدينا العديد من النقاط التي تكون local minima, وهناك Global minima واحدة وهدفنا هو الوصول لهذه النقطة الـ Global
 - لماذا هي غير محدبة؟
1. **التعقيد الهيكلي:** وجود طبقات متعددة (deep layers) يخلق تفاعلات غير خطية معقدة
 2. **دوال التفعيل غير الخطية:** استخدام دوال مثل Sigmoid, ReLU, Tanh يكسر الخطية ويخلق تضاريس معقدة
 3. **ملايين الأوزان:** وجود عدد هائل من المعاملات (الأوزان والانحيازات) يجعل مساحة البحث (فضاء البحث) ضخمة ومعقدة للغاية، مع عدد لا يحصى من الحدود الدنيا المحلية

الميزة (Feature)	التعلم الآلي التقليدي (Classic ML)	التعلم العميق (Deep Learning)
طبيعة المشكلة	غالباً محدبة (Convex)	دائماً تقريباً غير محدبة (Non-Convex)
الهدف	إيجاد الحل الأمثل (Global Optimum)	إيجاد حل جيد بما يكفي (Good Local Optimum)
الأدوات	محسّنات بسيطة (SGD) تكفي	محسّنات متقدمة ضرورية (Momentum Adam,)
التحدي	تصميم النموذج الصحيح	تجنب الاعتماد على حل سيء وتحسين الأداء

فائدة أريسيزية

■ نتحدث قليلاً عن نقص التخصيص (Underfitting)

أولاً: مثال بسيط (طالب كسول)

تخيل أن لديك طالباً (النموذج) وتريد أن ينجح في الامتحان النهائي (بيانات الاختبار)

■ حالة نقص التخصيص (Underfitting):

هذا الطالب كسول جداً ولم يدرس بشكل كافٍ. هو لم يفهم حتى المفاهيم الأساسية من الكتاب المدرسي (بيانات التدريب)

■ النتيجة :

سيرسب في الامتحانات التجريبية (بيانات التدريب) وسيرسب أيضاً في الامتحان النهائي (بيانات الاختبار)

■ الخلاصة :

النموذج فشل في تعلّم أبسط الأنماط، وبالتالي هو عديم الفائدة

ثانياً: التعريف :

■ نقص التخصيص (Underfitting) :

يعني أنني لم أصل الى أحسن بارامترات ويجب أن نغير أو نحسن حتى نصل لأدنى قاع وهي إحدى مشاكل ل SGD

gradients of the function that are represented by equation

لماذا التدرجات (gradients) لهذه الدالة كبيرة في اتجاه المحور y وصغيرة في اتجاه المحور x ؟
ولماذا لا تشير مباشرة إلى نقطة الصغرى (0,0) ؟

1. حساب التدرجات الرياضية

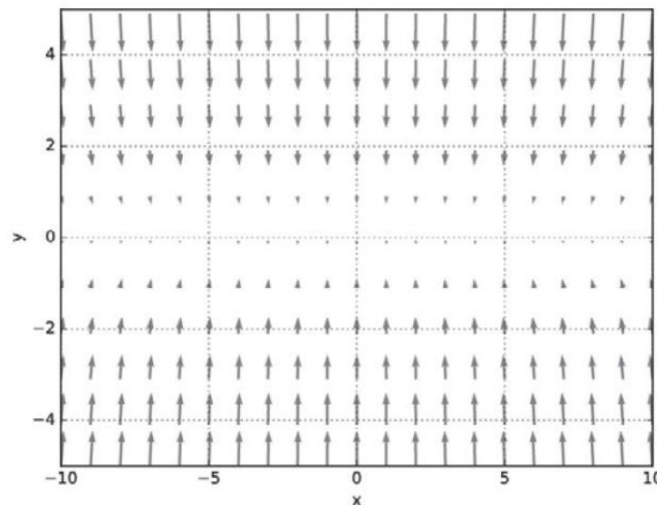
- أولاً، لنحسب التدرجات الرياضية للدالة $f(x, y) = (1/20)x^2 + y^2$
 - التدرج بالنسبة ل x : $\partial f / \partial x = (1/10)x$
 - التدرج بالنسبة ل y : $\partial f / \partial y = 2y$

2. لماذا التدرجات مختلفة في الحجم؟

من هذه المعادلات، نرى أن:

- بالنسبة للمحور y : المعامل هو 2، مما يعني أن التدرج في هذا الاتجاه يكون كبيراً
- بالنسبة للمحور x : المعامل هو 10/1، مما يعني أن التدرج في هذا الاتجاه يكون صغيراً جداً مقارنة بالاتجاه y
- مثل : لنحسب التدرج عند النقطة (5,5) فيكون التدرج عند $x = 0.5$ وعند $y = 10$
- وعند النقطة (6,6) فيكون التدرج عند $x = 0.6$ وعند $y = 12$
- إذا التدرج (المشتق) على المركبة y كبير بينما على المركبة x صغير أي الانتقال على y اسرع بكثير من الانتقال على x

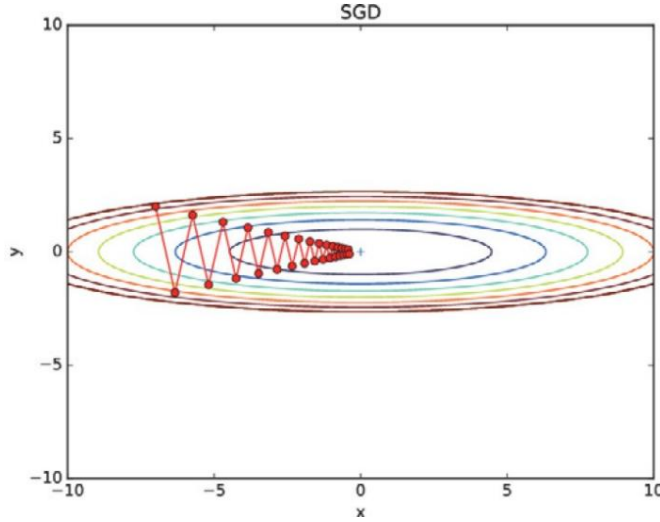
هذا يفسر لماذا الميل في اتجاه المحور y حاد، بينما في اتجاه المحور x يكون تدريجياً
لنرى الرسم :



وهي رسمة مركبات عكس ل gradient تم الضرب ب -1 لأن نريد اقل قيمة
نلاحظ أن:

- الأسهم (التدرجات) أطول في اتجاه المحور y (أعلى وأسفل)

- الأسهم أقصر في اتجاه المحور x (يمين ويسار)
- معظم الأسهم لا تشير مباشرة إلى المركز $(0,0)$ وإنما يتم الإشارة إلى مجموعة من القيم أي على أي قيمة يكون لها $y=0$, وهذا يعني أنني لا أشير إلى الـ global minima وهي النقطة $(0,0)$ وإنما تشير لأكثر من local minima
- لنرى الآن تحركات خوارزمية SGD بشكل متعرج :



كل دائرة تمثل قيمة ثابتة من قيم دالة الخسارة. ومع الانتقال من دائرة إلى أخرى باتجاه الداخل، فإن قيمة الخسارة تنخفض تدريجياً. وفي النهاية نصل إلى أصغر قيمة ممكنة، وهي تمثل نقطة واحدة فقط (نقطة الحد الأدنى)، ولكن في هذه الحالة نحن نتحرك بمسار متعرج و هدفنا هو التحرك بمسار مستقيم قدر الإمكان باتجاه الهدف

Momentum

خوارزمية ال Momentum هي أول خوارزمية حلت بعض عيوب خوارزميات ال SGD تخيل أنك تواجه مشكلة الوادي الممدود الذي ناقشناه سابقاً. بدلاً من أن يمشي المغامر خطوة بخطوة (مثل SGD) سنطلق كرة وندحرجها في هذا الوادي لن نتوقف لأن الكرة عم تزيد سرعتها كل ماتنزل

المكونات الفيزيائية للمومينتم

1. السرعة (Velocity - v)

- في SGD : كنا ننظر فقط إلى القوة الحالية (التدرج) ونتحرك بناءً عليها
- في المومينتم : نضيف مفهوم السرعة التي تراكمت من الخطوات السابقة.
- السرعة تتأثر بالقوة الحالية والسرعة القديمة

2. القوة (Force)

- القوة هنا هي **عكس التدرج** (negative gradient)
- التدرج يشير إلى الاتجاه الأكثر صعوداً، لذا نتحرك في عكسه
- هذه القوة تدفع الكرة في اتجاه معين

3. الاحتكاك (Friction - α)

- الاحتكاك هو معامل (عادة 0.9) يقلل من السرعة القديمة تدريجياً
- هذا يمنع الكرة من التسارع إلى ما لا نهاية
- يمثل مقاومة الهواء أو احتكاك الكرة مع الأرض

المعادلات الرياضية :

$$v \leftarrow \alpha v - \eta \frac{\partial L}{\partial W}$$

$$W \leftarrow W + v$$

الشرح :

$$v \leftarrow \alpha v - \eta \frac{\partial L}{\partial W}$$

السرعة الجديدة = (الاحتكاك × السرعة القديمة) + (القوة الجديدة)

$$W \leftarrow W + v$$

- تحديث الأوزان = الأوزان القديمة + السرعة المتراكمة

كيف يحل الموميتتم مشكلة المسار المتعرج؟

1. تراكم السرعة في الاتجاه الصحيح

- في اتجاه المحور x (المسطح) القوة صغيرة ولكنها ثابتة
- مع كل خطوة تتراكم السرعة في هذا الاتجاه، مما يسمح للكرة بالتحرك بشكل أسرع تدريجياً

2. إلغاء الحركة في الاتجاه الخاطئ

- في اتجاه المحور y (الحاد)، القوة كبيرة ولكنها متغيرة (يتغير اتجاهها)
- عندما تتحرك الكرة للأعلى ثم للأسفل، تتجه القوى في اتجاهين متعاكسين، مما يؤدي إلى إلغاء بعضها البعض

النتيجة: مسار أكثر سلاسة

بدلاً من المسار المتعرج (zigzag) SGD، تتحرك الكرة في مسار أكثر انحناءً وسلاسة نحو الهدف لا يختفي المسار المتعرج كلياً وإنما يخف قليلاً

هذا يشبه كيفية تحرك سيارة سباق في منعطف لا تتبع اتجاه الطريق بشكل فوري بل تستمر في الزخم الذي اكتسبته (الزخم يعني الكتلة * السرعة ولكن نحن لا يوجد لدينا كتلة)

ملخص الفوائد

1. تسريع التقارب: في الاتجاهات ذات التدرجات الثابتة الصغيرة

2. تقليل التذبذب: في الاتجاهات ذات التدرجات الكبيرة المتغيرة

3. تجاوز الحدود الدنيا المحلية الصغيرة: يمكن أن تساعد الكرة على القفز خارج الوديان الصغيرة

هذا هو جوهر تقنية المومنتم: إضافة زخم لعملية التحسين، مما يجعلها أكثر ذكاءً وفعالية من SGD البسيط. إنذار: خوارزمية مومنتم مثالية لحالة Saddle points فوراً بنزل بسبب وجود الزخم أيضاً تساعد في احتمال الخروج من local في حال كان معامل المومنتم عالي ولكن لديها مشكلة وهي أن عدد hyperparm زاد وهذا لا يعتبر جيد

implementation of Momentum

```

12 class Momentum:
13     def __init__(self, lr=0.01, momentum=0.9):
14         self.lr = lr
15         self.momentum = momentum
16         self.v = None
17
18     def update(self, params, grads):
19         if self.v is None:
20             self.v = {}
21             for key, val in params.items():
22                 self.v[key] = np.zeros_like(val)
23
24         for key in params.keys():
25             self.v[key] = self.momentum*self.v[key] - self.lr*grads[key]
26             params[key] += self.v[key]

```

الشرح :

1. دالة التهيئة __init__

- : self.lr = lr
نحفظ معدل التعلم (learning rate) هذا هو حجم الخطوة التي ستأخذها الكرة
- : self.momentum = momentum
نحفظ معامل الزخم (عادة 0.9) هذا هو معامل الاحتكاك الذي يبطئ الكرة تدريجياً
- : self.v = None
نهيئ متغير السرعة (velocity) على أنه فارغ
- التشبيه الفيزيائي: هذه هي مرحلة إعداد الكرة قبل إطلاقها:
- نحدد مدى سرعتها (learning rate)
- نحدد مدى احتكاكها بالأرض (momentum)
- نجهز مكاناً لتسجيل سرعتها (velocity)

2. دالة التحديث update

هذه هي الدالة الرئيسية التي تحدث في كل خطوة من تدريب الشبكة

الجزء الأول: تهيئة السرعة

..... : if self.v is None

ماذا يحدث هنا؟

- نتحقق إذا كانت هذه هي المرة الأولى التي نستدعي فيها دالة التحديث
 - إذا كانت كذلك، ننشئ قاموساً self.v بنفس هيكل params لأن كل بارميتر عم حدثه بدي حدثه حسب الزخم تبعو وحسب التدرج
 - لكل مفتاح في params، ننشئ مصفوفة أصفار بنفس الحجم
- التشبيه الفيزيائي: هذه هي مرحلة وضع الكرة في نقطة البداية:

- ننشئ سجل سرعة فارغ للكرة
- لكل جزء من الكرة (كل وزن في الشبكة)، نسجل سرعة ابتدائية تساوي صفر

الجزء الثاني: تحديث السرعة والمعاملات

.....: for key in params.keys()

ماذا يحدث هنا؟ لكل مفتاح (كل وزن في الشبكة):

$self.v[key] = self.momentum * self.v[key] - self.lr * grads[key]$

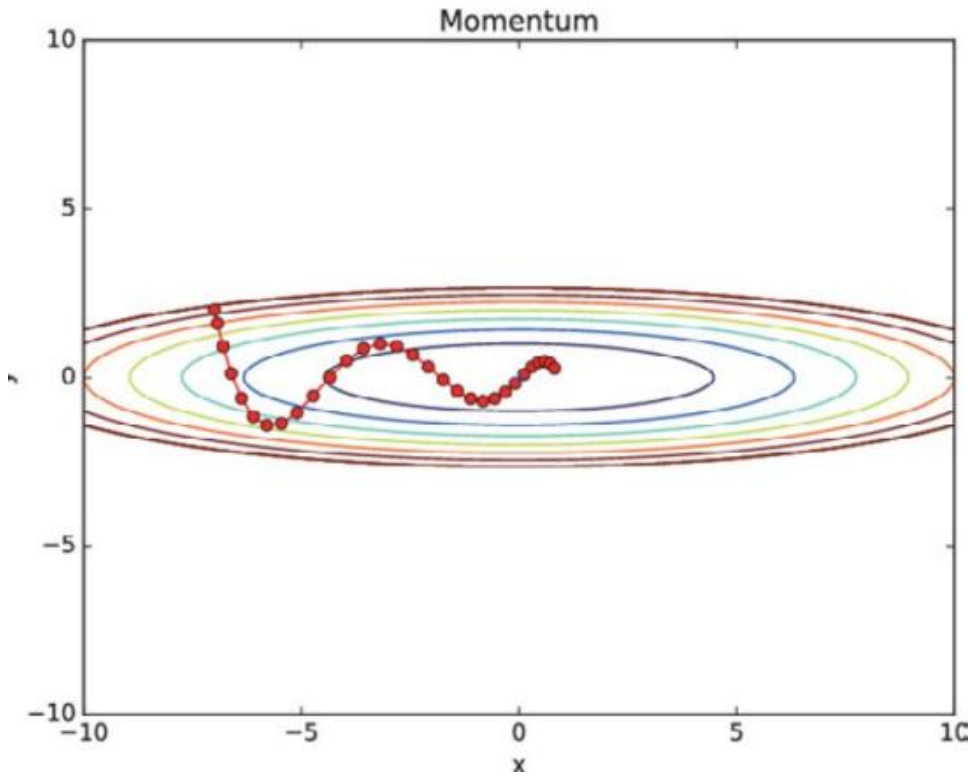
- نحسب السرعة الجديدة
- $self.momentum * self.v[key]$
- جزء من السرعة القديمة (مع الأخذ في الاعتبار الاحتكاك)
- $self.lr * grads[key]$
- القوة الجديدة (عكس التدرج مضروباً في معدل التعلم)

$params[key] += self.v[key]$

- نحدث الوزن الحالي بإضافة السرعة الجديدة
- التشبيه الفيزيائي: هذه هي مرحلة تحريك الكرة:
- 1. نحسب سرعتها الجديدة بناءً على:
 - سرعتها الحالية (مع تقليلها بسبب الاحتكاك)
 - 2. القوة الجديدة المؤثرة عليها (التدرج)
- نحدث موقع الكرة بناءً على سرعتها الجديدة

ملخص :

1. الإعداد الأولي: ننشئ كائن Momentum بتحديد معدل التعلم ومعامل الزخم
2. أول تحديث: ننشئ هيكل السرعة بنفس هيكل الأوزان، مع قيم ابتدائية تساوي صفر
3. كل تحديث لاحق:
 - نحسب السرعة الجديدة بناءً على السرعة القديمة والتدرج الحالي
 - نحدث الأوزان بناءً على السرعة الجديدة



الشرح :

لنعود لدالتنا $f(x, y) = (1/20)x^2 + y^2$ سنطلق كرة في الوادي الممدود بدلاً من ان يمشي المغامر خطوة خطوة

ماذا يحدث في اتجاه المحور السيني (x) ؟

- القوة صغيرة لكنها ثابتة: في هذا الاتجاه، التدرج (القوة) صغير جداً، لكنه دائماً في نفس الاتجاه (نحو المركز)
- تسارع مستمر: مع كل خطوة، تتراكم السرعة في هذا الاتجاه، الكرة تتحرك ببطء في البداية، ثم تزداد سرعتها تدريجياً
- النتيجة: الكرة تكتسب زخماً في اتجاه x وتتحرك بشكل أكثر فعالية

ماذا يحدث في اتجاه المحور (y) ؟

- القوة كبيرة لكنها متغيرة: في هذا الاتجاه، التدرج (القوة) كبير، لكنه يتغير اتجاهه باستمرار (مرة للأعلى، مرة للأسفل)
- إلغاء متبادل: عندما تتحرك الكرة للأعلى ثم للأسفل، تتجه القوى في اتجاهين متعاكسين، مما يؤدي إلى إلغاء بعضها البعض
- النتيجة: السرعة في اتجاه y تكون غير مستقرة، لكن الزخم المكتسب يساعد على تجاوز التذبذب

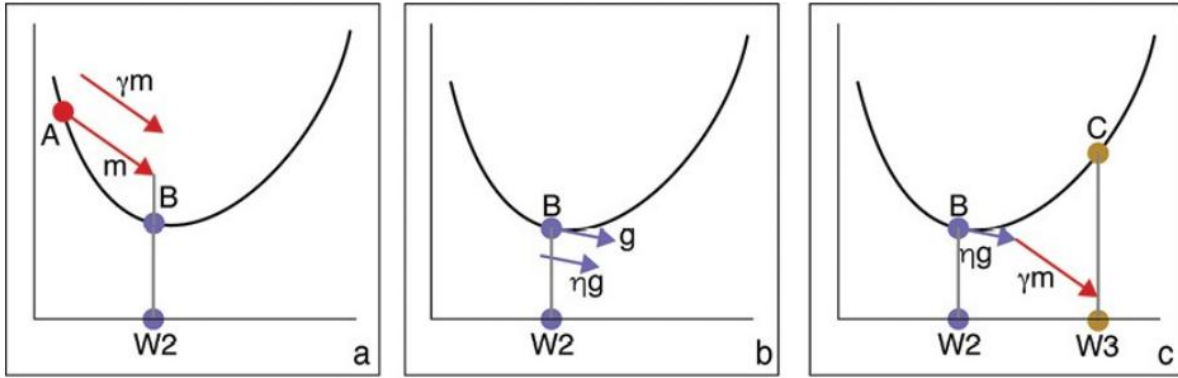
لماذا يقلل Momentum من الحركة المتعرجة؟

- في SGD :

- كل خطوة تعتمد فقط على التدرج الحالي
- الكرة تنسى أين كانت وكيف كانت تتحرك
- النتيجة: مسار متعرج واضح

في: Momentum

- الكرة تتذكر سرعتها السابقة
- في اتجاه x : السرعة تتراكم $\rightarrow$ حركة أسرع وأكثر سلاسة
- في اتجاه y : القوى المتعاكسة تلغي بعضها $\rightarrow$ حركة أقل تذبذباً



الشرح:

المخطط (a) مفهوم الزخم (Momentum Concept)

- النقطة A الحمراء: تمثل الموقع الحالي للمعاملات (الأوزان) في الشبكة العصبية
- النقطة B الزرقاء: تمثل الموقع الجديد الذي ستنتقل إليه المعاملات بعد تحديث واحد
- السهمان m و γm : هذان السهمان يمثلان قوة الزخم الفكرة هنا هي أن التحديث لا يعتمد فقط على الموقع الحالي (النقطة A) بل يأخذ في الاعتبار الاتجاه والسرعة التي كان يتحرك بها النموذج في الخطوة السابقة، هذا الزخم يساعد على تجاوز الاهتزازات الصغيرة والتوجه بشكل أكثر استقراراً نحو الحل الأمثل

المخطط (b) خوارزمية هبوط التدرج العشوائي (SGD)

- النقطة B الزرقاء: تمثل الموقع الحالي للمعاملات
- السهم g : يمثل متجه التدرج (gradient vector) حيث التدرج يشير إلى اتجاه أسرع زيادة في دالة الخسارة وبما أن هدفنا هو تقليل الخسارة، فنحن نريد التحرك في عكس هذا الاتجاه

- السهم ηg : يمثل خطوة التحديث الفعلية وهو متجه التدرج (g) مضروباً في معدل التعلم (η) هذا السهم يحدد حجم الخطوة التي سنخطوها في عكس اتجاه التدرج
 - النقطة W_2 : هي الموقع الجديد بعد تطبيق خطوة التحديث
- باختصار: الموقع الجديد = الموقع القديم - (معدل التعلم $\times$ التدرج) إنها خطوة مباشرة بناءً على ميل السطح الحالي فقط

المخطط (c) دمج الزخم والتدرج (Momentum Update)

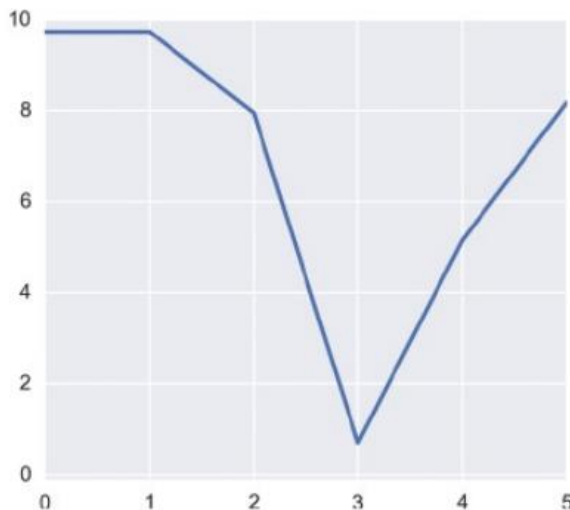
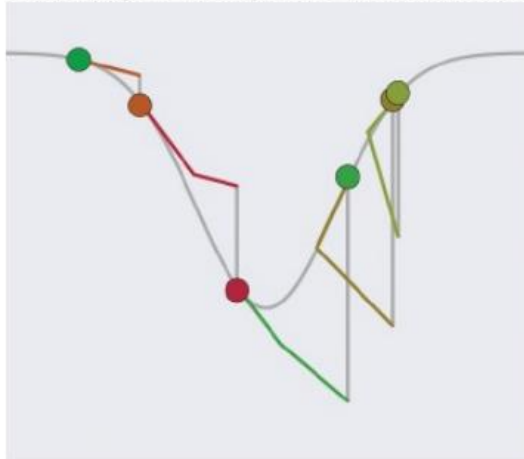
- النقطة C: هي نقطة البداية الموقع الحالي للمعاملات W_{old}
- السهم γm : يمثل جزء الزخم من التحديث، إنه متجه يمثل اتجاه الحركة في الخطوة السابقة، مضروباً في معامل الاحتكاك α
- السهم ηg : يمثل جزء التدرج من التحديث، هذه هي القوة الجديدة التي يؤثر بها التدرج الحالي على الحركة.
- السهم m : يمثل متجه التحديث الكلي، هذا هو حاصل جمع متجه الزخم (γm) ومتجه التدرج (ηg) إنه الاتجاه النهائي والسرعة النهائية التي سيتحرك بها النموذج
- النقطة W_3 : هي الموقع النهائي الجديد بعد تطبيق التحديث الكلي (m)
- $W_{new} = W_{old} + m$

باختصار: معادلة الزخم:

- السرعة الجديدة = (الاحتكاك $\times$ السرعة القديمة) + القوة الجديدة (التدرج)
- التحديث الكلي $(m) = \gamma m + \eta g$
- ثم يتم تطبيق هذا التحديث الكلي للوصول إلى الموقع الجديد W_3

تقليل معدل التعلم مع الزخم (Shrinking Learning Rate with Momentum) وتأثيرها على عملية تدريب الشبكة العصبية:

Shrinking Learning Rate with Momentum



الرسم البياني: مقارنة مسارات التقارب (Convergence Paths Comparison) :

هذه الرسم توضح كيف تتحرك خوارزميات التحسين المختلفة في فضاء المعاملات للوصول إلى الحل الأمثل (النقطة في المنتصف التي تمثل أدنى قيمة لدالة الخسارة)

- **الخطوط الملونة:** كل خط ملون يمثل مساراً مختلفاً
- **النقاط:** النقاط على كل خط تمثل خطوات التحديث المتتالية للمعاملات (الأوزان)
- **الهدف:** الهدف من كل هذه المسارات هو الوصول إلى المركز (أدنى نقطة في الوعاء)، والذي يمثل أفضل أداء للنموذج

ماذا نستفيد من هذا الرسم؟

- نرى أن جميع المسارات تتجه نحو الهدف، لكن بعضها قد يكون أكثر كفاءة وأقل تذبذباً من غيره
- استخدام الزخم (Momentum) يساعد على تجنب المسارات المتعرجة (Zigzag) ويوجه النموذج بشكل أفضل نحو الحل
- تقليل معدل التعلم يجعل الخطوات أصغر كلما اقتربنا من الهدف، مما يساعد على الاستقرار الدقيق عند الوصول إليه بدلاً من التجاوز

الرسم البياني: منحنى الخسارة (Loss Curve) مع مرور الوقت

هذه الرسم توضح كيف تتغير قيمة دالة الخسارة (Loss Function) بمرور الوقت (أو مع عدد مرات التدريب Epoch) هذا هو المقياس الأهم لمدى نجاح عملية التدريب

- **المحور العمودي (Y-axis):** يمثل قيمة الخسارة (الخطأ) كلما انخفض الرقم، كان أداء النموذج أفضل
- **المحور الأفقي (X-axis):** يمثل زمن التدريب أو عدد مرات التكرار (Iterations/Epochs)

لنفس شكل المنحنى:

1. الانخفاض الحاد في البداية: (The Sharp Initial Drop)

في بداية التدريب، يكون النموذج جاهلاً والخسارة عالية جداً. معدل التعلم يكون مرتفعاً، مما يسمح للنموذج باتخاذ خطوات كبيرة وسريعة للتعلم وتقليل الخطأ بشكل ملحوظ

2. التذبذبات: (The Oscillations or Wiggles)

هذه التموجات تحدث عادةً عند استخدام خوارزمية الزخم (Momentum) تخيل كرة تتدحرج في وعاء؛ عندما تقترب من القاع، قد تتخطاه قليلاً بسبب سرعتها (الزخم) ثم ترتد للخلف، وهكذا هذه التذبذبات هي تمثيل رياضي لهذه الحركة

3. تلاشي التذبذبات: (The Damping of Oscillations)

هذا هو الجزء الأهم والمتعلق بـ تقليل معدل التعلم كلما تقدمنا في التدريب، نقوم بتقليل قيمة معدل التعلم (Learning Rate Decay)

لماذا؟ معدل التعلم الأصغر يعني خطوات أصغر. عندما يقترب النموذج من الحل الأمثل، لا نريده أن يتخطاه ويرتد بقوة نريده أن يتحرك ببطء وثبات ليستقر تماماً عند أدنى نقطة ممكنة هذا التقليل التدريجي لمعدل التعلم هو ما يجعل التذبذبات أصغر وأصغر حتى تختفي تقريباً .

AdaGrad

المشكلة التي يحلها AdaGrad :

تخيل أنك تتعلم قيادة سيارة في طريقين مختلفين في نفس الوقت:

1. طريق جبلي وعراً جداً :ملء بالمنعطفات الحادة والتلال

2. طريق مستقيم وصحراوي :طويل ومستقيم ولا يوجد فيه أي عوائق.

لو استخدمت نفس قوة الدواس (نفس معدل التعلم η) في الطريقين، ماذا سيحدث؟

▪ في الطريق الجبلي: إذا ضغطت على دواسة البنزين بقوة، ستنزلق وتصطدم بالجبل مباشرة (هذا يسمى Divergence أو Overshooting)

▪ في الطريق الصحراوي: إذا ضغطت على الدواس بنفس القوة، ستصل ببطء شديد وممل هذا يسمى Slow Convergence

هذه هي مشكلة خوارزمية SGD العادية إنها تستخدم حجم خطوة واحد لجميع المعاملات (Parameters) ، وهو غير فعال لأن المعاملات المختلفة تتحرك في تضاريس مختلفة .

حل AdaGrad :

AdaGrad هو مثل سائق شديد الملاحظة يلاحظ حالة كل طريق على حدة

فكرته عبارة عن سؤال بسيط :

لماذا لا نجعل حجم الخطوة (معدل التعلم) يتكيف مع كل طريق على حدة؟

أي كل باراميتري له معدل تعلم خاص فيه

كيف يفعل ذلك؟

يقوم AdaGrad بتدوين ملاحظات أثناء القيادة هذه الملاحظات هي المتغير h

▪ المتغير h ذاكرة التدرج: هذا المتغير هو عبارة عن مذكرة أو سجل لكل معامل، في كل مرة نريد تحديث

معامل معين، ننظر في سجله h

▪ إذا كان المعامل يتغير دائماً بقيمة كبيرة (تدرج كبير)، هذا يعني أنه في طريق جبلي وعراً، نضرب هذا التدرج

الكبير في نفسه (نرفعه للتربيع) ونضيفه إلى h مع مرور الوقت، يصبح h كبيراً جداً لهذا المعامل أي عملية

حساب h هي تراكمية

▪ إذا كان المعامل يتغير بقيمة صغيرة (تدرج صغير)، هذا يعني أنه في طريق سهل، نضيف قيمة صغيرة جداً

إلى h ، فيبقى h صغيراً.

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{\sqrt{h}} \frac{\partial L}{\partial \mathbf{W}}$$

الخطوة الأولى: تحديث الذاكرة $(h) \mathbf{h} \leftarrow \mathbf{h} + (\partial L / \partial \mathbf{W}) \odot (\partial L / \partial \mathbf{W})$

▪ $\partial L / \partial \mathbf{W}$: هو التدرج أو انحدار الطريق في هذه الخطوة

▪ $\odot$: تعني الضرب العنصري (element-wise multiplication) ببساطة، نضرب التدرج في نفسه

▪ المعنى: يا h خذ قيمتك القديمة وأضف لها مربع انحدار الطريق اليوم، بهذه الطريقة، تتراكم صعوبة الطريق في الذاكرة h

▪ أي يمكن اعتبار أن h هي مربع التدرج في حال لا يوجد h سابقة

▪ لماذا نقوم بتربيع التدرج؟ لأننا نريد التخلص من الإشارة السالبة، لأن السالب سيغير من الاتجاه في حال كان التدرج -10 فالخطوة التالية ستصبح -10/1 و مضروبة بإشارة سالب بالتالي أصبحت الإشارة موجبة، أي أصبحنا نتحرك باتجاه التدرج ونحن نريد التحرك بعكس اتجاهه.

الخطوة الثانية: تحديث المعلمات $(W) \mathbf{W} \leftarrow \mathbf{W} - \eta * (1/\sqrt{h}) * \partial L / \partial \mathbf{W}$

▪ η : هو معدل التعلم الأولي (قوة الدواس)

▪ $1/\sqrt{h}$:

▪ للمعاملات التي كانت في طرق جبلية (قيم h كبيرة):

يكون $\sqrt{h}$ كبيراً وبالتالي $1/\sqrt{h}$ يكون صغيراً جداً

هذا يقلل بشكل فعال من معدل التعلم لذلك المعامل، مما يجعلها تأخذ خطوات صغيرة وحذرة

▪ للمعاملات التي كانت في طرق سهلة (قيم h صغيرة):

يكون $\sqrt{h}$ صغيراً، وبالتالي $1/\sqrt{h}$ يكون قريباً من 1

هذا يحافظ على معدل التعلم الأصلي لذلك المعامل، مما يسمح له بالاستمرار في التحرك بسرعة معقولة

الخلاصة:

AdaGrad هي خوارزمية ذكية تجعل كل معامل في الشبكة العصبونية يختار حجم خطوته الخاصة بناءً على تاريخه

▪ المعاملات التي شهدت تدرجات كبيرة في الماضي، يتم تقليل سرعتها أي تقليل حجم الخطوة

▪ المعاملات التي شهدت تدرجات صغيرة، تُترك لتواصل مسيرتها بسرعة أكبر أي ترك حجم الخطوة كبير

هذا يمنع الانزلاق (Overshooting) في الاتجاهات الحادة ويحسن من سرعة التقارب في الاتجاهات الهادئة، مما يجعل عملية التدريب بأكملها أكثر كفاءة واستقراراً.

الآن لنرى implement AdaGrad

```

31 class AdaGrad:
32
33     def __init__(self, lr=0.01):
34         self.lr = lr
35         self.h = None
36     def update(self, params, grads):
37         if self.h is None:
38             self.h = {}
39             for key, val in params.items():
40                 self.h[key] = np.zeros_like(val)
41         for key in params.keys():
42             self.h[key] += grads[key] * grads[key]
43             params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)
44

```

الشرح :

1. دالة __init__(مرحلة الإعداد الأولي)

- `self.lr = lr` : هنا نعطي قيمة معدل التعلم (Learning Rate)، وهو حجم الخطوة الأولية الذي سنبدأ به
- القيمة الافتراضية هي 0.01 تخيلها كقوة الضغط الأولية على دواصة البنزين
- `self.h = None` : هنا نجهز مذكرة فارغة اسمها h هذه المذكرة ستسجل فيها تاريخ حركة كل جزء في الشبكة العصبية
- في البداية هي فارغة (None)

2. دالة update مرحلة التحديث

هذه هي الدالة التي يتم استدعاؤها مرارًا وتكرارًا أثناء التدريب

- `params` : هي الموقع الحالي لجميع أجزاء الشبكة العصبية (الأوزان)
- `grads` : هي خريطة الانحدار (التدرج) التي تخبرنا بمدى انحدار الطريق عند كل جزء

...:if self.h is None

هذا الجزء يتحقق: هل المذكرة (h) لا تزال فارغة؟

إذا كانت فارغة، فإنه ينشئ صفحة فارغة لكل جزء في الشبكة العصبية ليبدأ بتدوين الملاحظات عليه `np.zeros_like(val)` تعني اصنع لي صفحة بنفس حجم هذا الجزء لكن مليئة بأصفار

...: for key in params.keys()

لدينا هنا خطوتان:

الخطوة الأولى: تسجيل الملاحظات في المذكرة

```
self.h[key] += grads[key] * grads[key]
```

لكل جزء في الشبكة، نأخذ انحدار الطريق ($grads[key]$) ونضربه في نفسه (نرفعه للتربيع)

ثم نضيف النتيجة إلى صفحته في المذكرة ($self.h[key]$)

المعنى: يا مذكرة، سجل لي مدى صعوبة الطريق اليوم لهذا الجزء إذا كان الطريق شديد الانحدار (قيمة كبيرة)، ستزداد قيمة المذكرة بسرعة

الخطوة الثانية: اتخاذ الخطوة

```
params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)
```

```
params[key] -= ...:
```

هذا هو التحديث الفعلي للموقع نحن ننقل الجزء إلى موقع جديد

```
: self.lr * grads[key]
```

هذه هي الخطوة العادية التي كانت ستأخذها الخوارزمية البسيطة (SGD)

```
: / (np.sqrt(self.h[key]) + 1e-7)
```

هنا AdaGrad نأخذ الخطوة العادية ونقسمها على الجذر التربيعي لما سجلناه في المذكرة

إذا كانت المذكرة ($self.h[key]$) تحتوي على أرقام كبيرة (طريق وعرة)، فإن القاسم سيكون كبيراً، وبالتالي الخطوة

النهائية ستكون صغيرة جداً (هذا هو الضغط على الفرامل)

إذا كانت المذكرة تحتوي على أرقام صغيرة (طريق سهل)، فإن القاسم سيكون صغيراً، وبالتالي الخطوة النهائية

ستظل كبيرة نسبياً

```
: + 1e-7
```

هذه مجرد طريقة أمان هي رقم صغير جداً (مثل 0.0000001) نضيفه لمنع القسمة على صفر، وهو خطأ رياضي.

تخيلها كقطعة مطاطية صغيرة تمنع دواسرة الفرامل من الالتصاق بالأرض تماماً

الخلاصة:

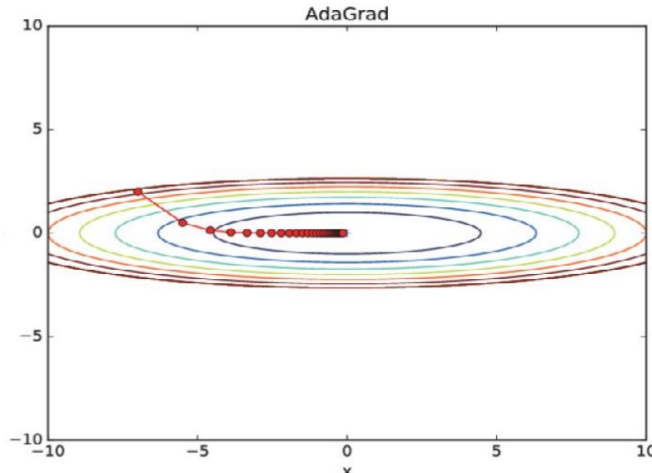
هذا الكود ببساطة هو تطبيق لفكرة السائق شديد الملاحظة

1. يجهز مذكرة (h) لتسجيل تاريخ كل طريق

2. في كل مرة، يسجل مدى وعورة الطريق في المذكرة

3. عندما يريد أن يخطو خطوة جديدة، ينظر إلى المذكرة إذا كان الطريق وعراً جداً، يضغط على الفرامل

(يجعل الخطوة أصغر) وإذا كان الطريق سهلاً، يكمل طريقه



الشرح:

تم حل مشكلة التدرج حيث كان التدرج على y كبير الان نأخذ جزء منه اما على x نأخذه كما هو .

والآن لنحدث عن Adam

Adam هي خوارزمية تحسين تجمع بين فكرتين أساسيتين:

1. Momentum الزخم: تعطي المعاملات زخماً، مثل كرة تتدحرج في وعاء، مما يساعدها على تجاوز العوائق الصغيرة.
2. AdaGrad التعلم التكيفي: تضبط حجم الخطوة (معدل التعلم) لكل معامل على حدة، مما يجعل التعلم أكثر كفاءة

كيف يبدو مسار التحديث؟

- مسار Adam يشبه حركة كرة تتدحرج في وعاء، تماماً مثل Momentum
- لكن الفرق الرئيسي: الحركات الجانبية (اليمين واليسار) للكرة في Adam أصغر بكثير

لماذا الحركات الجانبية أصغر؟

- هذه هي ميزة Adam، والسبب هو التعديل التكيفي لمعدل التعلم (الجزء المأخوذ من AdaGrad)
- ببساطة، Adam يكبح سرعة التحديث في الاتجاهات التي يكون فيها التدرج كبيراً
 - هذا يمنع الكرة من التذبذب أو الارتجاج بقوة بين جدران الوعاء، ويجعل مسارها أكثر استقامة وفعالية نحو القاع

أي في خوارزمية Adam يتم التحديث بناءً على v و m

المعاملات الفائقة (Hyperparameters)

لدى Adam ثلاثة معلمات أساسية:

1. α ألفا معدل التعلم. (Learning Rate): يتحكم بحجم الخطوة العام.
2. β_1 بيتا 1 معامل الزخم: يتحكم بمدى تأثير الاتجاهات السابقة، القيمة القياسية هي 0.9
3. β_2 بيتا 2 معامل التكيف: يتحكم بمدى تأثير معدل التعلم التكيفي، القيمة القياسية هي 0.999

لنشرح قليلاً عن بيتا 1 وبيتا 2 :

beta1 معامل الزخم:

$\beta_1 = 0.9$ يعني أن Adam يحتفظ بـ 90% من الزخم السابق m

- فقط 10% يأتي من التدرج الجديد gradient
- هذا يشبه إعطاء وزن أكبر للاتجاهات السابقة
- مثال: تخيل أنك تقود سيارة وتحتاج لتحديد اتجاهك:
- 90% من اتجاهك يعتمد على الاتجاه الذي كنت تسير فيه سابقاً
- 10% فقط يأتي من المعلومات الجديدة التي تراها الآن

beta2 معامل التدرج التربيعي:

$\beta_2 = 0.999$ يعني أن Adam يحتفظ بـ 99.9% من التدرجات التربيعية السابقة.

فقط 0.1% من المعلومات الجديدة تضاف في كل خطوة.

هذا يشبه AdaGrad لكن باستخدام مركب متناقص بدلاً من النمو المستمر تعني أن Adam يأخذ فكرة AdaGrad (تتبع تاريخ التدرجات) لكنه يحل مشكلتها الرئيسية عن طريق نسيان المعلومات القديمة تدريجياً، مما يسمح للنموذج بمواصلة التعلم بفعالية لفترة أطول.

📌 مثال :

تخيل أنك تحتفظ بسجل لسرعتك:

99.9% من سجل سرعتك يعتمد على السرعات السابقة

فقط 0.1% يأتي من سرعتك الحالية

لماذا هذه القيم؟

هذه القيم (0.9 و 0.999) تم اختيارها بعناية:

$\beta_1 = 0.9$: يعطي وزناً كافياً للزخم دون تجاهل المعلومات الجديدة

$\beta_2 = 0.999$: يحافظ على معلومات تاريخية كافية لضمان استقرار التعلم

هذه القيم تعمل بشكل جيد في معظم الحالات، ولهذا السبب أصبحت القيم الافتراضية في معظم تطبيقات Adam

```

48 class Adam:
49     def __init__(self, lr=0.001, beta1=0.9, beta2=0.999):
50         self.lr = lr
51         self.beta1 = beta1
52         self.beta2 = beta2
53         self.iter = 0
54         self.m = None
55         self.v = None
56     def update(self, params, grads):
57         if self.m is None:
58             self.m, self.v = {}, {}
59             for key, val in params.items():
60                 self.m[key] = np.zeros_like(val)
61                 self.v[key] = np.zeros_like(val)
62             self.iter += 1
63             lr_t = self.lr * np.sqrt(1.0 - self.beta2**self.iter) / (1.0 - self.beta1**self.iter)
64             for key in params.keys():
65                 self.m[key] += (1 - self.beta1) * (grads[key] - self.m[key])
66                 self.v[key] += (1 - self.beta2) * (grads[key]**2 - self.v[key])
67                 params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)
68

```

🔍 الشرح:

1. دالة التهيئة (init)

- **lr**: معدل التعلم، يحدد حجم الخطوة التي نتخذها لتحديث الأوزان
- **beta1**: معامل الزخم (عادة 0.9)، يحدد مدى تأثير التدرجات السابقة
- **beta2**: معامل التكييف (عادة 0.999)، يحدد مدى تأثير تربيع التدرجات
- **m, v**: متغيران سيتم استخدامهما لتخزين المعلومات التاريخية

2. دالة التحديث (update)

في المرة الأولى التي يتم فيها استدعاء الدالة، يتم تهيئة **m** و **v** بنفس أبعاد الأوزان ولكن بقيم صفرية

self.iter += 1

lr_t = self.lr * np.sqrt(1.0 - self.beta2self.iter) / (1.0 - self.beta1**self.iter)**

يتم تحديث معدل التعلم بشكل تكييفي مع كل تكرار

هذا يسمح بخطوات أكبر في البداية وأصغر مع اقترابنا من الحل

for key in params.keys():

▪ **m**: يتم تحديثه ليأخذ في الاعتبار التدرج الحالي والتدرجات السابقة مثل Momentum

▪ **v**: يتم تحديثه ليأخذ في الاعتبار تربيع التدرج الحالي والتدرجات السابقة مثل AdaGrad

params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)

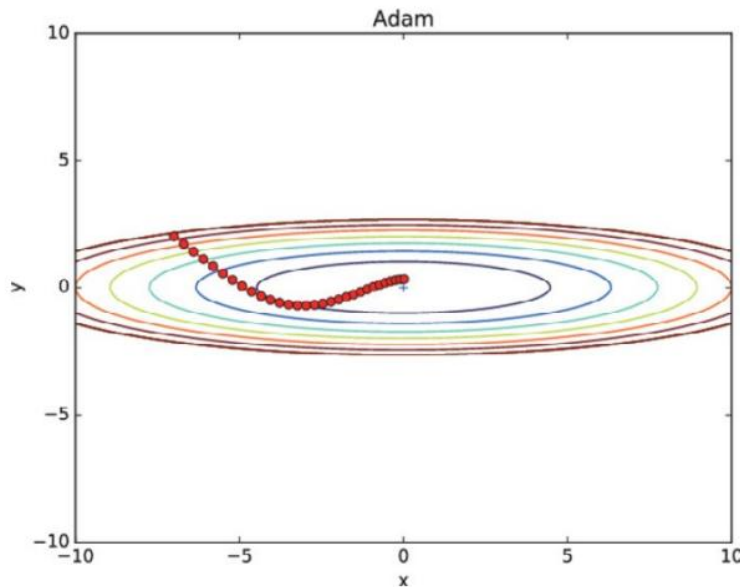
- يتم تحديث الأوزان باستخدام معدل التعلم المعدل (lr_t)
- يتم قسمة m على الجذر التربيعي $\sqrt{v}$ لتحديد حجم الخطوة المناسب
- يتم إضافة قيمة صغيرة ($1e-7$) لتجنب القسمة على صفر

✚ إذاً لدينا القوانين التالية :

الوظيفة	القانون الرياضي	السطر في الكود
تحديث الزخم (m)	$m = \beta_1 * m + (1 - \beta_1) * g$	<code>self.m[key] += (1 - self.beta1) * (grads[key] - self.m[key])</code>
تحديث التكيف (v)	$v = \beta_2 * v + (1 - \beta_2) * g^2$	<code>self.v[key] += (1 - self.beta2) * (grads[key]**2 - self.v[key])</code>
تصحيح الانحياز ومعدل التعلم	$\eta_{corrected} = \eta * \frac{\sqrt{1 - \beta_2^t}}{(1 - \beta_1^t)}$	<code>lr_t = self.lr * np.sqrt(1.0 - self.beta2**self.iter) / (1.0 - self.beta1**self.iter)</code>
التحديث النهائي للمعاملات (W)	$W = \frac{W - \eta_{corrected} * m}{(\sqrt{v} + \epsilon)}$	<code>params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)</code>

- الكود لدينا لا يحسب $gm_corrected$ $v_corrected$ بشكل منفصل، بل يدمج عملية التصحيح مباشرة في حساب معدل التعلم النهائي
- حيث ϵ هو رقم صغير جداً (مثل $1e-7$ لمنع القسمة على صفر)

لنرى شكل الخوارزمية :



في معظم الاحيان تكون خوارزمية Adam هي الافضل لكن في مسألتنا لم تكن كذلك وانما AdaGrad كانت الافضل

ملاحظة : كل مازاد عدد Hyperparameters كل ماكان هاد الشئ سيء

مثل ماذا نعتبره Hyperparameters ؟ مثل عدد الطبقات / عدد النيورونات في كل طبقة / معدل التعلم / دوال التنشيط

الخوارزمية	تشبيه بسيط	الحل الذي يقدمه	المشكلة التي يحلها
SGD (Stochastic Gradient Descent)	شخص يغلق عينيه ويحاول النزول من جبل، يخطو خطوة واحدة فقط بناءً على مكان قدمه الآن.	يتحرك خطوة واحدة فقط في عكس اتجاه التدرج الحالي، بسيط وسهل التنفيذ.	وُجدت لحل مشكلة الكلفة الحسابية والذاكرة عند التعامل مع بيانات كبيرة لكنها تعاني من الحكة المتعرجة (Zigzag) وبطء التقارب في المسائل المعقدة، لا تأخذ بالاعتبار التدرجات السابقة
Momentum (الزخم)	كرة تتدحرج في وعاء، لها زخم يبقيها تتحرك في نفس الاتجاه العام ويساعدها على تجاوز التلال الصغيرة	يضيف زخماً للتحديث، يجمع بين اتجاه التدرج الحالي واتجاه الخطوة السابقة، مما يساعد على تجاوز العقبات الصغيرة	مشكلة الحركة المتعرجة (Zigzag) وبطء SGD يصطدم بالعقبات الصغيرة ويتذبذب حولها
AdaGrad (Adaptive Gradient)	سائق ذكي يبطئ سرعته في المنعطفات الحادة (المعاملات ذات التدرج الكبير) ويزيدها في الطرق المستقيمة (المعاملات ذات التدرج الصغير)	ينشئ معدل تعلم مخصص لكل معامل، يقلل معدل التعلم للمعاملات التي تتغير كثيراً، ويبقيه كبيراً للمعاملات التي تتغير قليلاً	استخدام معدل تعلم واحد لجميع المعاملات غير فعال، بعض المعاملات تحتاج خطوات كبيرة، وأخرى تحتاج خطوات صغيرة
Adam (Adaptive Moment Estimation)	سائق سيارة رياضية، لديه نظام للحفاظ على السرعة (Momentum) ونظام تحكم تكيفي بالدواسات للتكيف مع الطريق (AdaGrad)	يجمع بين Momentum و AdaGrad. يستخدم ذاكرة لاتجاه التدرجات مثل Momentum وذاكرة لحجم التدرجات مثل AdaGrad بطريقة محسنة لا تتلاشى	جمع بين سرعة momentum وذكاء AdaGrad ومشكلة تلاشي معدل التعلم في AdaGrad على المدى الطويل

Initial Weight Values

لماذا القيم الأولية للأوزان مهمة جداً؟
تخيل أنك تبني منزلاً والأساس هو أول شيء تضعه، إذا كان الأساس ضعيفاً أو في مكان خاطئ، فإن المنزل بأكمله سيتأثر، حتى لو كانت بقية المواد ممتازة

الأوزان الأولية (Initial Weights) هي أساس الشبكة العصبونية:

- إذا اخترنا قيماً سيئة في البداية، قد لا تتعلم الشبكة بشكل صحيح أبداً، أو قد تتعلم ببطء شديد
- فاختيار القيم الأولية الصحيحة يمكن أن يسرع عملية التعلم بشكل كبير

لماذا لا نضع كل الأوزان = 0؟

إذا أردنا أوزاناً صغيرة، فلماذا لا نجعلها كلها صفراً؟

هذه فكرة سيئة لنشرح لماذا:

المشكلة هي التماثل (Symmetry):

تخيل أن لديك فريقاً من العمال متطابقين تماماً (كلهم يرتدون نفس الملابس ولديهم نفس المهارات) إذا أعطيتهم جميعاً نفس المهمة بنفس التعليمات، فماذا سيحدث؟

سيقومون جميعاً بنفس الشيء بنفس الطريقة

هذا يحدث بالضبط في الشبكة العصبونية إذا كانت جميع الأوزان صفراً:

1. في مرحلة الانتشار الأمامي (Forward Propagation):

عندما ترسل البيانات إلى الشبكة، جميع الخلايا العصبونية في الطبقة الثانية ستستقبل نفس القيمة بالضبط، لأن الأوزان التي تصلها من الطبقة الأولى كلها صفر

2. في مرحلة الانتشار الخلفي (Backpropagation):

عندما يحين وقت التعلم وتحديث الأوزان، ستستقبل جميع الخلايا العصبونية في الطبقة الثانية نفس إشارة التعلم وبالتالي ستقوم بتحديث أوزانها بنفس الطريقة وب نفس القيمة

النتيجة الغير مرضية:

- ستظل جميع الأوزان متطابقة دائماً
- لن تتعلم خلية عصبية شيئاً مختلفاً عن أختها وأنا أريد ان تتعلم كل خلية شيئاً مختلف
- كأن لديك 100 عامل ولكنهم يعملون كشخص واحد فقط

الحل: كسر التماثل (Symmetry Breaking)

لحل هذه المشكلة، يجب أن نبدأ ب قيم عشوائية

عندما نعطي كل وزن قيمة أولية عشوائية صغيرة (مثل $np.random.randn(...) * 0.01$)، فإننا نكسر التماثل

- الآن كل خلية عصبية تبدأ من نقطة مختلفة قليلاً عن أختها
- عند استقبال البيانات، ستستقبل كل خلية قيماً مختلفة قليلاً
- عند التعلم، ستقوم كل خلية بتحديث وزنها بطريقة فريدة خاصة بها

- أيضاً تهيئة الاوزان بقيم كبيرة قريبة من 1 و -1 ليس جيد لماذا ؟

لأنه سوف ينتج لدينا حالة اشباع saturation سنتحدث عنها لاحقاً

الخلاصة :

- نحن بحاجة إلى قيم أولية عشوائية لضمان أن كل جزء من الشبكة العصبية يمكن أن يتعلم شيئاً مختلفاً ومميزاً
- إذا بدأنا جميعاً من نفس النقطة (مثل الصفر)، فسنظل جميعاً متطابقين ولن نتعلم أي شيء جديد
- العشوائية هي التي تسمح بالتنوع والتعلم الفعال

Distribution of Activations in the Hidden Layers

ما هي التنشيطات (Activations) ؟

- تخيل أن الشبكة العصبية هي سلسلة من الموظفين الذين يتناقلون معلومة
- عندما يعطي الموظف الأول المعلومة للموظف الثاني، يقوم الموظف الثاني بـ معالجة هذه المعلومة ثم يمررها للثالث

التنشيطات هي شكل المعلومة بعد معالجتها من قبل كل موظف (طبقة)

إذاً، تنشيطات الطبقة المخفية:

هي المخرجات التي تنتجها الطبقات الوسطى في الشبكة العصبية بعد تطبيق دالة التنشيط (مثل sigmoid) عليها
لماذا نراقب توزيع هذه التنشيطات؟

مراقبة توزيع هذه التنشيطات تشبه مراقبة صحة تدفق المعلومات في الشبكة

- إذا كان التدفق صحيحاً: المعلومات متنوعة وتنتشر بشكل جيد عبر الطبقات
- إذا كان التدفق مريضاً: المعلومات قد تتجمع في أماكن معينة أو تختفي تماماً
هذا التوزيع يخبرنا ما إذا كانت الشبكة تتعلم بشكل فعال أم لا

التجربة الاولى: ماذا يحدث عند استخدام أوزان عشوائية كبيرة؟

لنقوم بتجربة بسيطة لنرى ما يحدث عندما نستخدم قيماً أولية كبيرة للأوزان

```

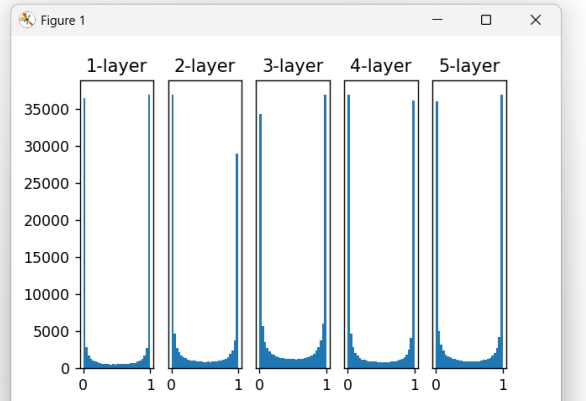
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6
7 def ReLU(x):
8     return np.maximum(0, x)
9
10 def tanh(x):
11     return np.tanh(x)
12
13 input_data = np.random.randn(1000, 100)
14 node_num = 100
15 hidden_layer_size = 5
16 activations = {}
17 x = input_data
18

```

```

--
19 for i in range(hidden_layer_size):
20     if i != 0:
21         x = activations[i-1]
22         w = np.random.randn(node_num, node_num) * 1
23         a = np.dot(x, w)
24         z = sigmoid(a)
25         activations[i] = z
26 for i, a in activations.items():
27     plt.subplot(1, len(activations), i+1)
28     plt.title(str(i+1) + "-layer")
29     if i != 0:
30         plt.yticks([], [])
31     plt.hist(a.flatten(), 30, range=(0,1))
32 plt.show()
33

```



الشرح:

▪ : def sigmoid(x)

تحويل أي رقم إلى قيمة بين 0 و 1

▪ : def ReLU(x)

ترجع القيمة إذا كانت موجبة و 0 إذا كانت سالبة

▪ : def tanh(x)

دالة tanh تشبه sigmoid لكنها ترجع قيماً بين -1 و 1

ملاحظة: سنستخدم حالياً دالة sigmoid فقط، وسنستخدم بقية الدوال بتجارب أخرى

إعداد معاملات الشبكة والبيانات:

▪ : input_data = np.random.randn(1000, 100)

▪ يتم إنشاء بيانات إدخال عشوائية

▪ هذه البيانات هي 1000 عينة

▪ كل عينة لها 100 ميزة (أو 100 خلية عصبية في طبقة الإدخال)

▪ البيانات موزعة حسب التوزيع الطبيعي (متوسط 0 وانحراف معياري 1)

▪ : node_num = 100

▪ يحدد عدد الخلايا العصبية في كل طبقة مخفية (100 خلية)

▪ : hidden_layer_size = 5

▪ يحدد عدد الطبقات المخفية (5 طبقات)

▪ : activations = {}

▪ يتم إنشاء قاموساً (dictionary) فارغاً لتخزين مخرجات (تنشيطات) كل طبقة

▪ : x = input_data

▪ يتم تهيئة المتغير x ليحمل بيانات الإدخال، والتي ستكون مدخلات الطبقة الأولى

بناء الشبكة وتمرير البيانات:

- : for i in range(hidden_layer_size)
للمرور عبر كل طبقة من الطبقات الخمس المخفية
- : if i != 0: x = activations[i-1]
في الطبقة الأولى (i=0)، تكون المدخلات هي input_data
في الطبقات اللاحقة تكون المدخلات هي مخرجات الطبقة السابقة (activations[i-1])
- : w = np.random.randn(node_num, node_num) * 1
هنا المشكلة يتم إنشاء مصفوفة الأوزان w لكل طبقة
الأوزان عشوائية ولكنها كبيرة نسبياً لأن ضربنا ب 1
- : a = np.dot(x, w)
يتم ضرب مصفوفة المدخلات x في مصفوفة الأوزان w هذه هي العملية الخطية في الشبكة العصبية
- : z = sigmoid(a)
يتم تطبيق دالة sigmoid على النتيجة a للحصول على مخرجات الطبقة (التنشيطات)
- : activations[i] = z
يتم تخزين مخرجات (تنشيطات) الطبقة الحالية في القاموس activations

رسم النتائج

- for i, a in activations.items():
يمر على كل طبقة وتنشيطاتها المخزنة
- plt.subplot(1, len(activations), i+1):
يتم انشاء شبكة فرعية (subplot) لكل طبقة لرسمها بجانب بعضها البعض
- plt.title(str(i+1) + "-layer"):
يتم وضع عنوان لكل مخطط باسم الطبقة
- plt.hist(a.flatten(), 30, range=(0,1)):
يتم رسم مدرج تكراري (histogram) لقيم التنشيطات في كل طبقة
- a.flatten():
يحوّل مصفوفة التنشيطات إلى متجه واحد
- 30:
يحدد عدد الأعمدة في المدرج التكراري
- range=(0,1):
يحدد نطاق المدرج التكراري من 0 إلى 1 وهو نطاق مخرجات دالة sigmoid
- plt.show():

آلية العمل بشكل مختصر؟

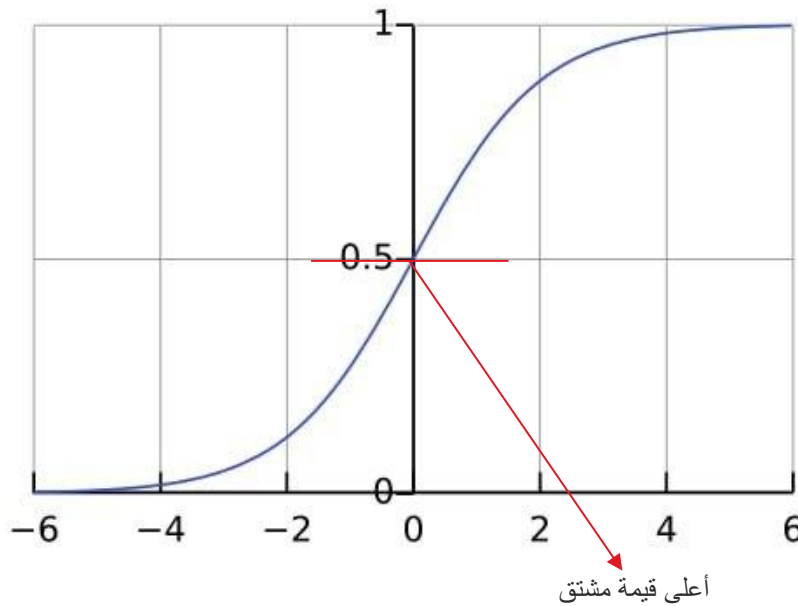
1. إنشاء بيانات: يولد 1000 عينة من بيانات الإدخال بشكل عشوائي
2. بناء شبكة: يبني شبكة عصبية بسيطة من 5 طبقات، كل طبقة بها 100 خلية عصبية
3. اختيار الأوزان: يقوم بتهيئة الأوزان بقيم عشوائية من توزيع طبيعي بانحراف معياري كبير (1)
4. مرور البيانات: يمرر البيانات عبر الشبكة طبقة تلو الأخرى
5. رسم النتائج: يرسم مدرج تكراري (histogram) يوضح لنا كيف تبدو قيم التنشيطات في كل طبقة

النتيجة: مشكلة اختفاء التدرج (Gradient Vanishing)

لماذا هذا سيء؟

تذكر أن دالة sigmoid هي دالة على شكل حرف S:

$$f(x) = 1 / (1 + e(-x))$$



- تضغط الدخل لقيم بين 0 و 1
- إذا كان الدخل كبير جدا - < الخرج = 1
- إذا كان الدخل صغير جدا - < الخرج = 0
- إذا كان الدخل متوسط - < الخرج بين 0 و 1
- إذًا عندما تكون مدخلات دالة سيغمويد كبيرة جدًا أو صغيرة جدًا، تكون المخرجات قريبة جدًا من 1 أو 0
- في هذه المناطق (عند 0 و 1)، يكون ميل الدالة (التدرج) قريبًا من الصفر
- حيث أعلى قيمة للمشتق هي 0 وقيمته 0.25 : نعوض في المشتق

$$\hat{f}(x) = f(x) \cdot (1 - f(x))$$

$$= 1 / (1 + e(-0)) \times (1 + 1 / (1 + e(-0))) = 0.25$$

المشكلة الكبرى:

- في الانتشار الخلفي (Backpropagation)، يتم استخدام هذا الميل لتحديث الأوزان
- إذا كان الميل صفراً تقريباً، فهذا يعني أن إشارة التعلم ستكون ضعيفة جداً أو معدومة
- مثل :

عندما يكون الدخل كبير يكون الخرج = 1 وبالتعويض بالمشتق :

$$\hat{f}(x) = 1 \times (1 - 1) = 0$$

عندما يكون الدخل صغير يكون الخرج = 0 وبالتعويض بالمشتق :

$$\hat{f}(x) = 0 \times (1 - 0) = 0$$

- هذه المشكلة تسمى اختفاء أو تلاشي التدرج (Gradient Vanishing) :
- مع كل طبقة إضافية تصبح إشارة التعلم أضعف وأضعف، حتى تختفي تماماً في الطبقات العميقة أي وصلت لحالة الاشباع ف العقد السابقة لن تتعلم شيء
- هذا يمنع الشبكة من التعلم بشكل صحيح

باختصار:

استخدام أوزان أولية كبيرة يؤدي إلى تنشيطات تتجمع عند 0 و 1، مما يجعل التدرجات تختفي، ويمنع الشبكة من التعلم

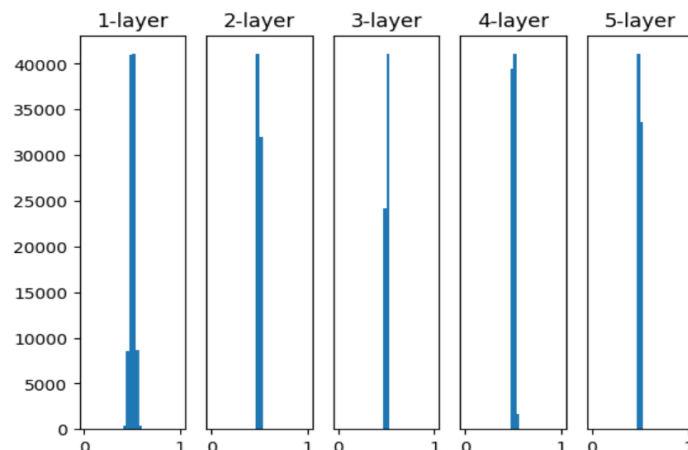
التجربة الثانية: ماذا يحدث عند استخدام أوزان صغيرة جداً؟

إذا كانت الأوزان الكبيرة تسبب مشكلة، فماذا عن جعلها صغيرة جداً؟
الكود نفسه ولكن قمنا بتغيير سطر تهيئة الأوزان :

`w = np.random.randn(node_num, node_num) * 0.01`

بدلاً من استخدام انحراف معياري 1، استخدمنا انحراف معياري صغير جداً وهو 0.01

لنرى شكل المدرج التكراري :



النتائج الجديدة: مشكلة التمثيل المحدود

- التنشيطات في جميع الطبقات: أصبحت متجمعة بشكل كبير حول القيمة 0.5
- المشكلة الجديدة: لم تعد هناك مشكلة اختفاء التدرج (Gradient Vanishing)، لأن 0.5 هي نقطة يكون فيها ميل دالة sigmoid عند أقصى حد له

لكن ظهرت مشكلة جديدة: التمثيل المحدود (Limited Representation)

لماذا التمثيل المحدود مشكلة؟

تخيل أن لديك فريقاً من 100 خبير، وكلهم متطابقون في رأيهم إذا سألتهم سؤالاً فسيجيبون جميعاً بنفس الإجابة تقريباً

هذا ما يحدث في الشبكة العصبية:

1. جميع الخلايا العصبية تنتج نفس الشيء: عندما تكون جميع التنشيطات متجمعة حول 0.5، فهذا يعني أن جميع الخلايا العصبية في الطبقة تخرج قيماً متشابهة جداً
2. لا فائدة من التعدد: إذا كانت 100 خلية عصبية تخرج نفس القيمة تقريباً، فما الفائدة من وجود 100 خلية؟ يمكن لخلية واحدة فقط أن تقوم بنفس العمل
3. فقدان التنوع: الشبكة العصبية قوية لأنها يمكنها تعلم ميزات مختلفة في خلايا عصبية مختلفة إذا كانت جميع الخلايا متشابهة، فإنها تفقد قدرتها على تعلم تمثيل متنوع للبيانات

بإختصار: الشبكة أصبحت غير فعالة لأنها لا تستغل إمكانياتها الكاملة لأن جميع أجزائها تعمل بنفس الطريقة

ما هو الحل؟

توزيع مناسب للتنشيطات

ماذا يعني توزيع مناسب؟

يعني أننا نريد التنشيطات أن تكون متنوعة ولكن ليس متطرفة

- ليست متطرفة (مثل الحالة الأولى): لا نريدها أن تتجمع عند 0 و 1 وتسبب اختفاء التدرج
- ليست متطرفة (مثل الحالة الثانية): لا نريدها أن تتجمع عند 0.5 وتسبب تمثيلاً محدوداً

نحن نريد توزيعاً متنوعاً، حيث تنتج الخلايا العصبية مجموعة واسعة من القيم، مما يسمح للشبكة بتعلم ميزات غنية ومختلفة

هذا هو السبب في أن اختيار الأوزان الأولية الصحيحة (مثل Xavier و He initialization) مهم جداً، لأنه يضمن أن يكون توزيع التنشيطات صحيحاً من البداية، مما يسمح للشبكة بالتعلم بكفاءة.

Xavier initializer(glorot)

ما هي تهيئة (Xavier Initialization) Xavier ؟

بعد أن رأينا مشكلتين:

أوزان كبيرة ← تنشيطات عند 0 و 1 ← اختفاء التدرج

أوزان صغيرة جداً ← تنشيطات عند 0.5 ← تمثيل محدود

جاء (Xavier) بحل ذكي ومتوازن فكرته هي :

دعنا نجعل حجم الأوزان يعتمد على عدد الخلايا العصبية في الطبقة

كيف تعمل تهيئة Xavier؟

كلما زاد عدد الخلايا العصبية التي تدخل إلى طبقة ما، كلما كان يجب أن تكون الأوزان أصغر

لماذا؟ تخيل أنك في حفلة، وكل شخص في الغرفة يهمس في أذنك معلومة

▪ إذا كان هناك 5 أشخاص فقط، فستكون المعلومات التي تتلقاها معقولة

▪ إذا كان هناك 1000 شخص يهمسون في أذنك في نفس الوقت، فستكون المعلومات ضخمة جداً

ومشوشة!

في الشبكة العصبية، الشيء نفسه يحدث:

▪ كل خلية عصبية في الطبقة التالية تستقبل إشارة من جميع الخلايا العصبية في الطبقة السابقة

▪ إذا كانت هناك العديد من الخلايا العصبية في الطبقة السابقة، فإن الإشارة الإجمالية ستكون كبيرة جداً

حل Xavier :

يجعل الأوزان أصغر كلما زاد عدد الخلايا العصبية، لضمان أن الإشارة الإجمالية ليست كبيرة جداً ولا صغيرة جداً

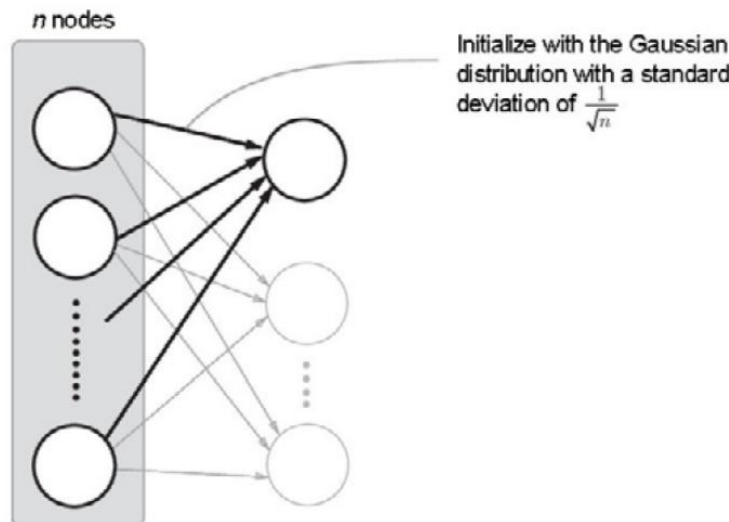
القاعدة (ببساطة)

بدلاً من استخدام أوزان عشوائية بانحراف معياري ثابت (1 أو 0.01)، يتم سحب الأوزان من توزيع طبيعي بمتوسط 0 ولكن

الانحراف المعياري يكون :

الانحراف المعياري = $1 / \sqrt{n}$ الجذر التربيعي لعدد الخلايا العصبية في الطبقة السابقة

$$\frac{1}{N} \text{ أو } \frac{1}{\sqrt{N}}$$



$w = \text{np.random.randn}(\text{node_num}, \text{node_num}) / \text{np.sqrt}(\text{node_num})$

- هنا node_num هو عدد الخلايا العصبية (100 في هذه الحالة)
- هذا يعني أن الأوزان أصغر من الحالة الأولى (الانحراف المعياري 1)
- ولكنها أكبر من الحالة الثانية (الانحراف المعياري 0.01)

النتائج: التوزيع المثالي

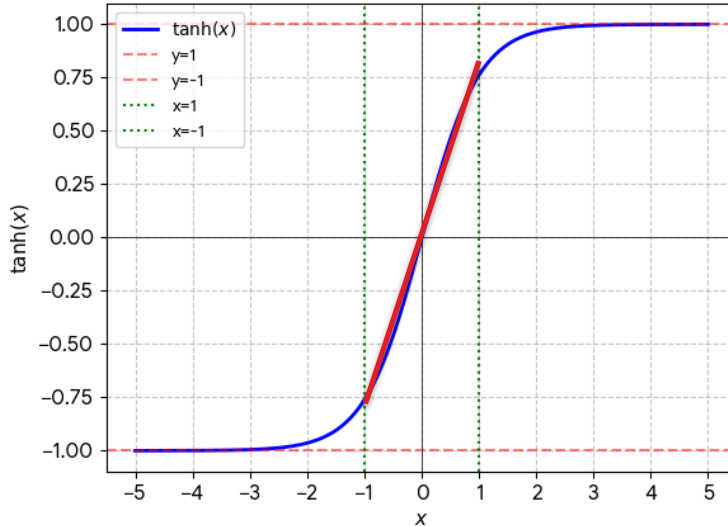
عندما نستخدم تهيئة Xavier، تكون النتائج رائعة:

- التنشيطات موزعة بشكل جيد :
 - بدلاً من التجمع عند 0 و 1 أو 0.5، تنتشر التنشيطات عبر نطاق واسع (من 0.2 إلى 0.8 تقريباً)
 - لا يوجد اختفاء للتدرج: لأن التنشيطات ليست عند 0 و 1
 - لا يوجد تمثيل محدود: لأن التنشيطات ليست كلها عند 0.5
- النتيجة: الشبكة العصبية تعمل بكفاءة عالية، البيانات تنتقل بين الطبقات بشكل متناسق، وكل خلية عصبية يمكن أن تتعلم شيئاً مختلفاً ومفيداً

لماذا تعمل Xavier بشكل أفضل مع دالة $\tanh$ بدلاً من sigmoid ؟

لماذا؟

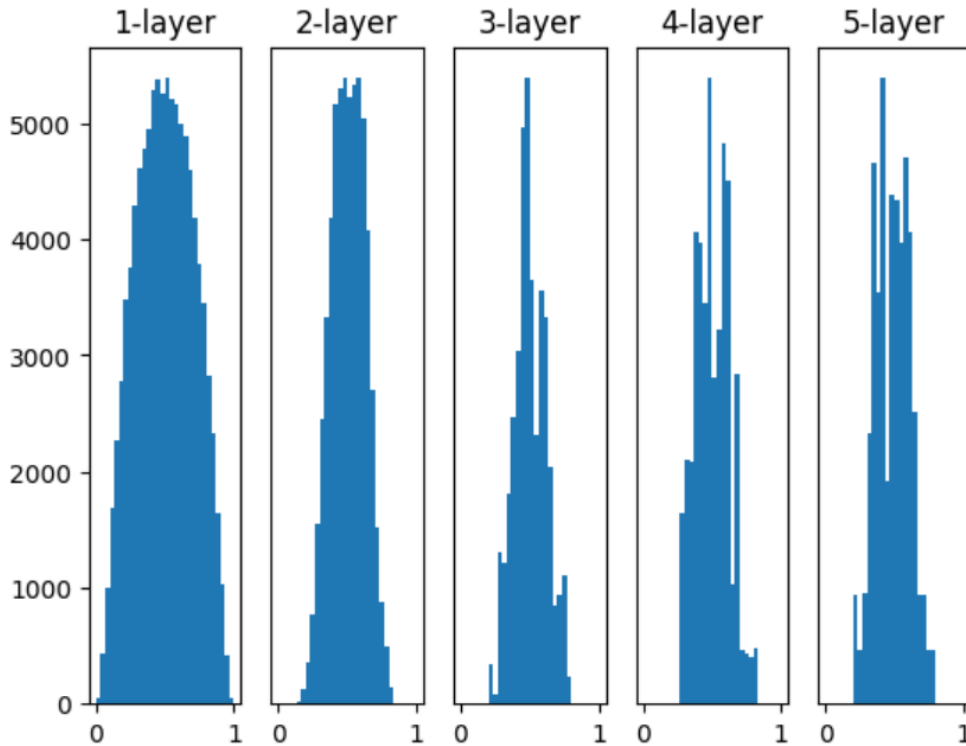
- دالة sigmoid :
- مخرجاتها بين 0 و 1، وهي ليست متمركزة حول الصفر متمركزة حول (0,0.5)
- دالة $\tanh$:
- مخرجاتها بين -1 و 1، وهي متمركزة حول الصفر (0, 0)



حيث أعلى قيمة للمشتق هي 1
وكلّ ماكان gradient أعلى كلّ ماكان افضل
ملاحظة:

لماذا التمرکز حول الصفر مهم؟

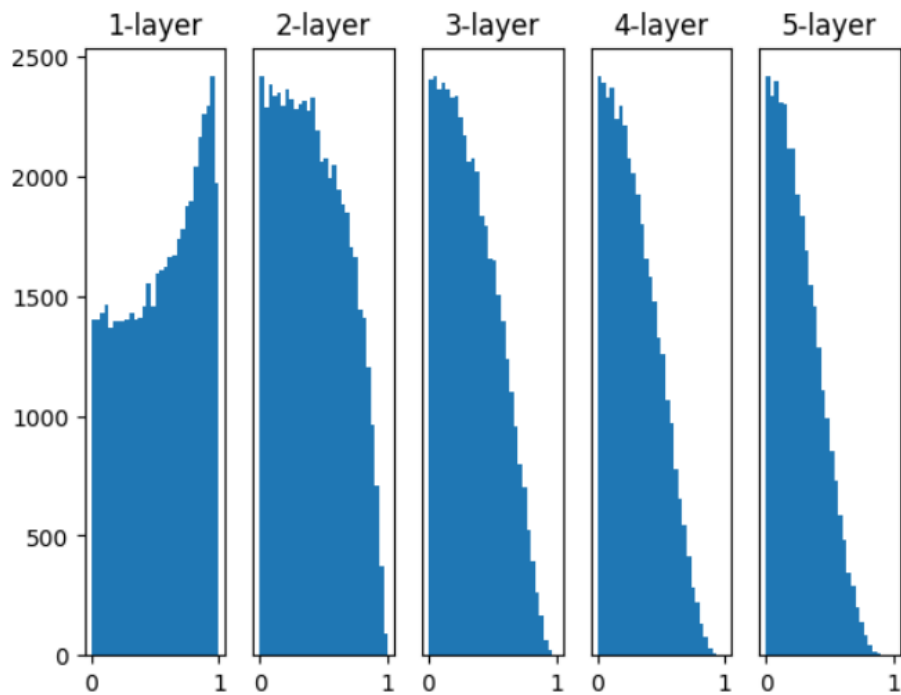
- عندما تكون البيانات متركزة حول الصفر، فإنها تساعد الشبكة على التعلم بشكل أسرع وأكثر استقرارًا
- التوزيع الجرسى الذي نحصل عليه مع $\tanh$ هو مثالي للتدريب الفعال باختصار $Xavier + \tanh =$ زوج مثالي
- $Xavier$ توفر توزيعًا جيدًا للتنشيطات، و $\tanh$ تحافظ على هذا التوزيع بشكل مثالي عبر الطبقات
- هذه هي الطريقة التي تستخدمها معظم أطر التعلم العميق الحديثة (مثل TensorFlow و PyTorch) بشكل افتراضي
- لنرى الفرق بشكل المخرج التكراري من حيث استخدام $Xavier$ مع $\tanh$ و sigmoid مع sigmoid :



الشرح:

: $Xavier + \text{sigmoid}$

حل جيد جداً، يجعل البيانات تنتشر وتتعلم بفعالية، ولكن مع بعض التشويه الطفيف في الطبقات العميقة .
مع $\tanh$:



الشرح:

: Xavier + tanh

حل أفضل، يجعل البيانات تنتشر وتتعلم بفعالية، ويحافظ على شكلها أفضل بسبب التوازن .

Initial Weight Values for ReLU The Xavier initializer

1. أولاً: مشكلة Xavier مع دالة ReLU

- تذكر أن تهيئة Xavier رائعة لدوال مثل sigmoid و tanh لأنها متماثلة حول الصفر
- لكن دالة ReLU مختلفة

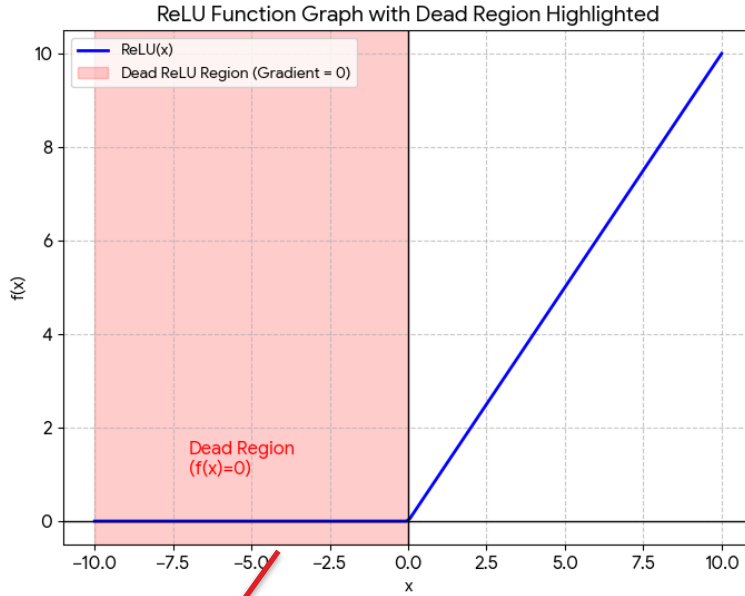
ما هي مشكلة ReLU ؟

دالة ReLU: $\max(0, x)$ مامشتق الدالة ؟

- 1 في حال $x > 0$
- 0 في حال $x \leq 0$

أي عندما يكون الرقم موجباً، تتركه كما هو وإذا كان سالباً، تحوله إلى صفر

فالمشكلة هو احتمال أن يكون نصف القيم أصفاراً، بالتالي هذه القيم لن تتعدل (لن تتعلم) حتى لو تغير الدخل x



Dead node

dead node لن نتعلم شيء ولن استفيد منها حتى لو غيرت الأوزان لن نتعلم
مثلاً اخترنا قيمة كبيرة للانحياز ولتكن -1000 مهما قمنا بالتعديل على قيمة w سيبقى الخرج في ReLU صفر لان ناتج $w x + b$ هو سالب

لماذا هذا يسبب مشكلة لـ Xavier ؟

Xavier مصممة بافتراض أن البيانات تتدفق في كلا الاتجاهين (الموجب والسالب) لكن ReLU يقتل نصف البيانات (الجزء السالب) اي مجموعة من العقد تكون بلا فائدة

مثال:

- لدينا أنبوب ماء
- Xavier مصمم لضمان تدفق الماء بسلاسة
- ReLU مثل وضع سد في منتصف الأنبوب، مما يسمح للماء بالتدفق في اتجاه واحد فقط
- هذا يقلل من كمية الماء التي تمر
- لهذا السبب، نحتاج إلى طريقة مختلفة لتهيئة الأوزان عند استخدام ReLU

2. ثانياً: الحل: تهيئة (He Initialization) He

هنا يأتي دور (He) :

إذا كانت ReLU تقتل نصف البيانات، فلنعزز الجزء المتبقي

كيف يعمل؟ احتمال الوقوع بعقد ميتة هو 50٪ وبالتالي يجب تكبير التباين اي :

▪ Xavier: الانحراف المعياري $\sigma = 1 / \sqrt{n}$

▪ He: الانحراف المعياري $\sigma = \sqrt{2/n}$

لماذا لم نستخدم $1 / \sqrt{n}$ ؟

- في حال لدينا $w_1 \cdot x_1 + w_2 \cdot x_2$ وكانت قيمة w_1 اصغر من صفر وقيمة w_2 اكبر من صفر وكانت w_2 هي week positive هاد الشي زاد احتمالية العودة للوقوع بالسالب (نتيجة الجمع النهائية) لذلك نحن نريد ان تكون strong positive لذلك لم نستخدم $1 / \sqrt{n}$ وإنما $\sqrt{2 / n}$
- بمعنى أي تعديل بسيط على الاوزان سيقوم بإرجاع w_2 بالمنطقة السالبة
- هذا التعزيز البسيط يضمن أن تتدفق البيانات بقوة أكبر عبر الشبكة، مما يعوض البيانات المفقودة في الجزء السالب

3. ثالثاً: التجربة: مقارنة الثلاث طرق مع ReLU

تجربة لمقارنة ثلاث طرق لتهيئة الأوزان عند استخدام ReLU :

1. الانحراف المعياري = 0.01 (صغير جداً)

- النتيجة:
- التنشيطات تصبح صغيرة جداً مع كل طبقة ($0.0396 \rightarrow 0.00290 \rightarrow \dots \rightarrow 9.46e-7$)
- المشكلة :
- موت (Dying ReLU) الإشارة تضعف وتختفي تماماً

مثال :

كأنك تهمس في أذن شخص، ثم يهمس في أذن التالي بصوت أضعف، حتى لا يسمع أحد شيئاً في النهاية الشبكة لا تتعلم شيئاً.

2. تهيئة Xavier

- النتيجة:
- تعمل بشكل أفضل من الحالة الأولى، لكن التنشيطات تبدأ بالانكماش مع تقدم الطبقات
- المشكلة:
- يعود ظهور مشكلة اختفاء التدرج (Gradient Vanishing) ببطء

مثال :

كأن لديك بطارية ضعيفة تعمل في البداية، لكنها تفقد شحنتها مع كل خطوة

3. تهيئة (He Initialization) He

- النتيجة:
- هي الأفضل توزيع التنشيطات يظل منتشرًا ومتناسقًا عبر جميع الطبقات
- لماذا هو الأفضل؟
- لأن التعزيز الذي يضيفه يضمن بقاء الإشارة قوية ومستقرة

مثال:

كأن لديك إشارة راديو قوية وواضحة تسمعها بنفس الوضوح في الغرفة الأولى والغرفة الخامسة

الخلاصة:

- إذا كنت تستخدم دالة ReLU (أو أي من أشكالها)، استخدم تهيئة He
 - إذا كنت تستخدم دالة S-curve مثل sigmoid أو tanh، استخدم تهيئة Xavier
- هذه القاعدة البسيطة تضمن أن تبدأ شبكتك العصبية بالطريقة الصحيحة وتتجنب الكثير من المشاكل الشائعة في التدريب

ملاحظات سريعة :

- قيمة $\tanh$ عند $\tanh$ ل vanishing
- عند $\tanh$: عند مجال معين قريب من الصفر يكون الميل تقريباً خطي (أي شكل $\tanh$ تقريباً خطي اي دخله يساوي خرجه وحيكون التدرج قيمته قريبة من الواحد)
- بالنسبة ل Gradient :
- أحيانا موجب وأحيانا سالب عند $\tanh$ يعني عم اتحرك بأكتر من اتجاه لإنها متناظرة بالنسبة للصفر اما عند سيغمويد بس موجب عم نتحرك باتجاه واحد
- إذا فالبحث عن ل optimal value باستخدام $\tanh$ أفضل من سيغمويد ولكن يوجد مشكلة وهي عند وصوله ل large pos/neg أي عند 1 و -1 يكون وصل لحد الاشباع فلن يتعلم شيء أي أنه وصل ل Gradient = 0
- تعقيد الشبكة في ReLU أقل لأن في مجموعة من العقد ReLU + useless مالح يوصل لل saturation
- يوجد مشكلة مشتركة بين سيغمويد و $\tanh$ تسمى saturation

سنتحدث الآن عن saturation

التشبع يعني أن دالة التنشيط تصل إلى نقطة لا تستجيب فيها لتغير المدخلات بعد الآن
كيف يحدث ذلك؟

بعض دوال التنشيط لديها مناطق مشبعة هذه هي المناطق التي يكون فيها المدخل (الرقم القادم للدالة) إما كبيراً جداً أو صغيراً جداً

لنأخذ مثال دالة Sigmoid التي تخرج دائماً قيماً بين 0 و 1:

- إذا أعطيتها رقماً كبيراً جداً (مثل 100)، ستكون النتيجة قريبة جداً من 1
- إذا أعطيتها رقماً أكبر (مثل 1000)، ستظل النتيجة قريبة جداً من 1 الفرق شبه معدوم
- نفس الشيء يحدث مع الأرقام السالبة الكبيرة جداً (مثل -100 و -1000)، ستكون النتيجة قريبة جداً من 0

- في هذه المناطق (عندما يكون المدخل كبيراً جداً أو صغيراً جداً)، تكون الدالة مشبعة وتتوقف عن التفاعل

لماذا التشبع مشكلة كبيرة جداً؟

- الشبكة تتعلم عن طريق حساب التدرج (Gradient) ، الذي هو الميل في دالة التنشيط.
- التدرج (الذي يخبر الشبكة كيفية تحديث الأوزان) يكون قريباً من الصفر لا يساوي تماماً صفر وإنما قريب
- عندما يكون التدرج صفراً، لا يحدث تعلم، الأوزان لا تُحدَّث، والشبكة تجمد وتتوقف عن التعلم في تلك الطبقة هذه المشكلة الشهيرة تسمى تلاشي التدرج (Vanishing Gradient)
- فالحل: كما تحدثنا سابقاً بدل من تهيئة الأوزان بقيم عشوائية من المجال $[-1, 1]$ حيث كان التوزيع طبيعي بمتوسط 0 وتباين 1 نأخذ قيم قريبة من الصفر أي متوسط 0 وتباين أقل من نصف أي $\frac{1}{\sqrt{N}}$ هي الطريقة التي أسمينها gloriot .

مقارنة بين saturation و Dead Neuron :

التشبع (Saturation) مشكلة Tanhg Sigmoid

- مع أن قيمة المشتق تكون قريبة جداً من الصفر، إلا أنها يمكن أن تتغير أثناء الانتشار الخلفي (Backpropagation)، يتم ضرب إشارة التعلم في رقم صغير جداً، مما يجعلها ضعيفة جداً
- الخلية العصبية لا تزال تتعلم، ولكن بوتيرة بطيئة جداً لدرجة أنها تصبح غير فعالة، هي لم تمت، لكنها شبه متوقفة عن العمل

الخلية الميتة (Dead Neuron) مشكلة ReLU :

- أثناء الانتشار الخلفي، يتم ضرب إشارة التعلم في صفر
- هذا يعني أن إشارة التعلم التي تصل إلى هذه الخلية العصبية هي صفر
- وبالتالي لن يتم تحديث أوزانها على الإطلاق
- الخلية تتوقف عن التعلم تماماً لهذه البيانات، إذا استمر هذا الوضع، تموت الخلية ولا تتعلم أي شيء مرة أخرى

الميزة	Saturation	Dead
الدوال المتأثرة	sigmoid, tanh	ReLU
المُحفِّز (Trigger)	مدخلات كبيرة جداً (موجبة أو سالبة)	أي مدخل سالب
قيمة الخرج	قريب جداً من الحد (مثل 0 أو 1)	صفر تماماً
قيمة التدرج	قريب جداً من الصفر	صفر تماماً
تأثير التعلم	التعلم يصبح بطيئاً جداً	التعلم يتوقف تماماً

Batch Normalization Algorithm

حلت العديد من المشاكل سوف نتكلم عنها

- تخيل أنك تبني شبكة عصبية مثل بناء منزل متعدد الطوابق
- كل طابق يمثل طبقة في الشبكة، والمعلومات تنتقل من طابق لآخر

ما هي مشكلة الشبكات العصبية العادية؟

عندما تنتقل المعلومات بين الطوابق، قد تتغير طبيعة هذه المعلومات بشكل كبير هذا مثل أن تمرر رسالة من شخص لآخر، وبعد عدة أشخاص قد تتغير الرسالة تمامًا

ماذا يفعل التطبيع الدفعي؟

التطبيع الدفعي يشبه مُنظّم معلومات يوضع في كل طابق من المبنى وظيفته:

1. يأخذ المعلومات القادمة من الطابق السابق
2. يعيد تنظيمها بحيث تكون طبيعية (متوسطها صفر وانحرافها المعياري واحد)
3. يضيف بعض التعديلات الخاصة لتناسب الطابق التالي

بإختصار Batch Norm هو طبقة مثل باقي الطبقات عادةً تكون قبل دالة التنشيط او بعد حيث الورقة الاصلية للبحث كانت تضعها قبل ولكن العديد من التجارب أثبتت ان وضع هذه الطبقة بعد دالة التنشيط تعطي نتائج افضل لنرى المخطط التالي الذي يوضح أنه تم وضع Batch Norm بين كل طبقة والتالية الذي شبهناه ب مُنظّم معلومات حيث تم وضعها قبل دالة التنشيط :



لماذا هذا مفيد؟

- **تسريع التعلم**: بدلاً من أن تضطر الشبكة لتعلم كيفية التعامل مع المعلومات المتغيرة باستمرار، فإن التطبيع الدفعي يجعل المعلومات أكثر استقراراً، مما يسمح للشبكة بالتعلم بشكل أسرع
- **أقل اعتماداً على القيم الأولية**: عادةً، لا يجب أن تكون حذرًا جدًا عند اختيار القيم الأولية للأوزان مع التطبيع الدفعي، يمكنك أن تكون أقل قلقاً لأنه يصحح المشاكل أثناء التدريب
- **تقليل الإفراط في التخصيص**: Overfitting : يساعد التطبيع الدفعي في منع الشبكة من حفظ بيانات التدريب بشكل مفرط، مما يجعلها تعمل بشكل أفضل على بيانات جديدة

كيف يعمل ؟

1. يأخذ مجموعة من البيانات (دفعة صغيرة)
2. يحسب المتوسط والانحراف المعياري لهذه المجموعة
3. تطبيع البيانات بحيث يكون متوسطها صفر وانحرافها المعياري واحد
4. يضيف بعض التعديلات γ و β التي تتعلمها الشبكة أثناء التدريب

لنرى معادلات تطبيع الدفعات :

1. حساب المتوسط والانحراف المعياري وتطبيع القيم :

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

1. المعادلة الاولى : تحسب متوسط قيم الدفعة (batch) الحالية من البيانات

- μ_B هو متوسط الدفعة
- m هو عدد العينات في الدفعة
- x_i هي قيمة كل عينة

2. المعادلة الثانية : تحسب التباين (أو الانحراف المعياري المربع) للدفعة

- σ_B^2 هو تباين الدفعة
- تقيس مدى تشتت القيم عن المتوسط

3. المعادلة الثالثة Standardization: تطبع القيم الأصلية بحيث يكون متوسطها صفر وانحرافها المعياري واحد

- $\hat{x}_i$ هي القيمة المطبوعة
- نطرح المتوسط ونقسم على الانحراف المعياري
- ε هو رقم صغير جداً (مثل 7^{-10}) لمنع القسمة على صفر
- بعد أن قمنا بتطبيع القيم (جعل متوسطها صفر وانحرافها المعياري واحد)، لكن أحياناً لا نريد أن تكون البيانات (0 و 1) دائماً، فربما نحتاج الشبكة بشكل مختلف لتتعلم بشكل أفضل أي نحتاج إلى إعادة تحجيمها وإزاحتها (scale and shift) لتناسب الشبكة بشكل أفضل
- حيث كل طبقة كما تحدثنا لديها مجموعة من البارامترات القابلة للتعلم فأصبح لدينا الآن لدينا اوزان و انحياز وغاما وبيتا
- أيضاً يجب ان يكون شكل كل من غاما وبيتا نفس شكل الانحياز أي متجه مثلاً في حال كان شكل متجه الانحياز (5,) يجب ان يكون شكل غاما وبيتا ايضاً (5,)

لنرى معاً المعادلة التي تساعدنا في تحقيق ذلك :

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

γ (غاما): هو معامل التحجيم (scaling parameter)

- يبدأ بقيمة 1 افتراضياً
- يتعلم أثناء التدريب القيمة المناسبة
- إذا كانت $\gamma > 1$ ، فإنه يوسع التوزيع
- إذا كانت $\gamma < 1$ ، فإنه يضغط التوزيع

β (بيتا): هو معامل الإزاحة (shift parameter)

- يبدأ بقيمة 0 افتراضياً
 - يتعلم أثناء التدريب القيمة المناسبة
 - يحرك التوزيع يمينا أو يساراً
- بماذا تفيد هذه المعادلة ؟

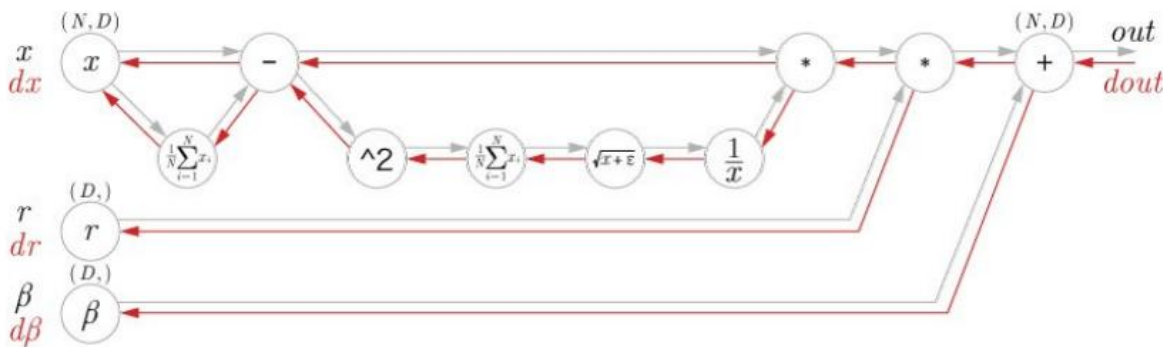
1. في المرونة أي : التطبيق (متوسط صفر وانحراف معياري واحد) بما انه قد لا يكون دائماً الأفضل للشبكة , هذه المعادلة تزيح التوزيع الطبيعي الاصلي وتجعله يناسب دالة التنشيط الموجودة اي تحويل من طبيعي standard لطبيعي لكن ليس standard اي تمنح الشبكة مرونة لاختيار التوزيع الأنسب
مثل : دالة سغمويد تصبح البيتتا عندها $\frac{1}{\sqrt{N}}$
2. أيضاً في التحسين أي : تسمح للشبكة بتعلم أفضل توزيع للبيانات في كل طبقة

٤ مثال سريع :

لديك بيانات موزعة على شكل جرس (منحنى غاوسي):

1. أولاً نطبعها لتصبح في المنتصف تماماً (متوسط صفر) وعرضها معين (انحراف معياري واحد)
 2. ثم نقرر إذا كان هذا العرض مناسباً أم نحتاج لتوسيعه أو تضيقه (هذا دور γ)
 3. وأخيراً نقرر إذا كان مكان الجرس مناسباً أم نحتاج لتحريكه يمينا أو يساراً (هذا دور β)
- بهذه الطريقة، تحافظ الشبكة على استقرار التدريب مع الحفاظ على المرونة اللازمة للتعلم الأمثل

- يمكننا تمثيل تطبيع الدفعات باستخدام computational graph الذي تعلمناه سابقاً:



الشرح:

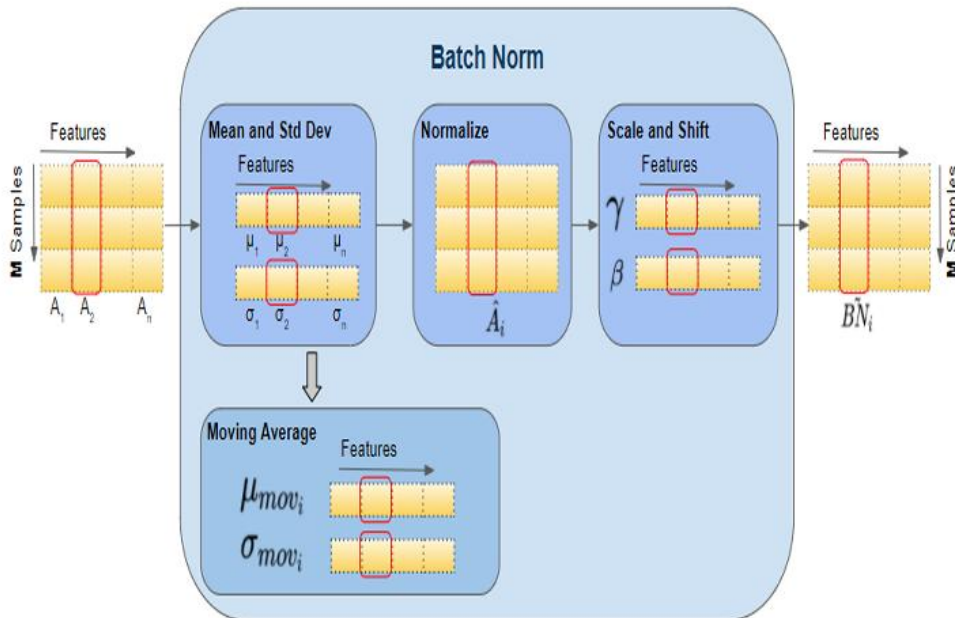
[في مرحلة المرور الامامي:](#)

- عقدة الطرح (-): لطرح المتوسط
- عقدة التربيع (2^8): لحساب التباين
- عقدة القسمة (\): لتقسيم البيانات وجعلها موحدة
- عقدة الضرب: لتطبيق gamma
- عقدة الجمع: لإضافة beta

في المرور العكسي:

نبدأ بـ dy (المشتق القادم من الطبقة التالية) ونعود للخلف:

1. **عقدة الجمع:** عندما نصل لعقدة إضافة beta، الاشتقاق يمر كما هو ببساطة beta هو مجموع كل القيم القادمة
2. **عقدة الضرب:** عندما نصل لعقدة الضرب في gamma، نطبق قاعدة السلسلة Chain Rule، الاشتقاق dy يتأثر بقيمة البيانات التي ضربت فيها
3. **عقدة التوحيد (Normalization):** التدرج يجب أن يمر عبر المتوسط والتباين
لماذا هذا معقد؟ لأن كل عنصر في الدفعة (Batch) يؤثر على المتوسط، وبالتالي يؤثر على بقية العناصر لذا، الاشتقاق هنا يتوزع على كل العناصر بالتساوي
إذاً: **الإدخال dout:** الخطأ القادم من الأعلى
المخرجات dx : (الخطأ الممرر للطبقة السابقة) بالإضافة لتحديثات gamma و beta



لنرى الآن تطبيق ذلك : حيث يتم تنظيم البيانات القادمة من الطبقة السابقة قبل أن تصل للطبقة التالية

```

78 class BatchNormalization:
79     def __init__(self, gamma, beta, momentum=0.9, running_mean=None, running_var=None):
80         self.gamma = gamma
81         self.beta = beta
82         self.momentum = momentum
83         self.running_mean = running_mean
84         self.running_var = running_var
85         self.batch_size = None
86         self.xc = None
87         self.std = None
88         self.dgamma = None
89         self.dbeta = None
90
91     def forward(self, x, train_flg=True):
92         return self.__forward(x, train_flg)
93
94 def __forward(self, x, train_flg):
95     if self.running_mean is None:
96         N, D = x.shape
97         self.running_mean = np.zeros(D)
98         self.running_var = np.zeros(D)
99
100     if train_flg:
101         mu = x.mean(axis=0)
102         xc = x - mu
103         var = np.mean(xc**2, axis=0)
104         std = np.sqrt(var + 10e-7)
105         xn = xc / std
106         self.batch_size = x.shape[0]
107         self.xc = xc
108         self.xn = xn
109         self.std = std
110         self.running_mean = self.momentum * self.running_mean + (1-self.momentum) * mu
111         self.running_var = self.momentum * self.running_var + (1-self.momentum) * var
112     else:
113         xc = x - self.running_mean
114         xn = xc / ((np.sqrt(self.running_var + 10e-7)))
115
116     out = self.gamma * xn + self.beta
117     return out
118
119 def backward(self, dout):
120     return self.__backward(dout)
121
122 def __backward(self, dout):
123     dbeta = dout.sum(axis=0)
124     dgamma = np.sum(self.xn * dout, axis=0)
125     dxn = self.gamma * dout
126     dxc = dxn / self.std
127     dstd = -np.sum((dxn * self.xc) / (self.std * self.std), axis=0)
128     dvar = 0.5 * dstd / self.std
129     dxc += (2.0 / self.batch_size) * self.xc * dvar
130     dmu = np.sum(dxc, axis=0)
131     dx = dxc - dmu / self.batch_size
132     self.dgamma = dgamma
133     self.dbeta = dbeta
134     return dx

```

الشرح:

في دالة التهيئة:

▪ β و γ : هما المعاملان اللذان تتعلمهما الشبكة لإعادة ضبط البيانات بعد تطبيعها

- momentum بسرعة تحديث الذاكرة حيث 0.9 تعني الحفاظ على 90% من ذاكرتك القديمة و إضافة 10% من معلومات الدفعة الجديدة
- `running_mean` و `running_var` هما الذاكرة التي تخزن المتوسط والتباين العام لكل البيانات

في دالة المرو الامامي :

هنا لدينا منطقتين مختلفتين بناءً على قيمة `train_flg` هل نحن ندرب أم نختبر؟

(a) في وضع التدريب (`train_flg=True`) :

- ينظر للدفعة الحالية فقط: يحسب المتوسط (μ) والانحراف المعياري (σ) لهذه الدفعة الصغيرة من البيانات
- يُنظَّم البيانات: يجعل متوسطها صفراً وانحرافها المعياري واحداً ($x_n = x_c / \sigma$)
- يُعَدَّل البيانات: يضرب البيانات المنظمة في `gamma` ويضيف (`out = ... beta`)
- يُحدَّث ذاكرته: يدمج إحصائيات الدفعة الحالية مع ذاكرته `running_mean`, `running_var` باستخدام `momentum`

(b) في وضع الاختبار (`train_flg=False`) :

1. يتجاهل الدفعة الحالية: لا يحسب أي إحصائيات جديدة
2. يستخدم ذاكرته فقط: يستخدم `running_mean` و `running_var` التي تعلمها من كل بيانات التدريب لتنظيم البيانات الجديدة
3. يُعَدَّل البيانات: يطبق نفس `gamma` و `beta` التي تعلمها

في المرو الخلفي :

عندما يأتي خطأ من الطبقة التالية (`dout`) ، يجب على المُنظَّم أن يحدد مسؤوليته عن هذا الخطأ

1. يحسب الخطأ الخاص به: يحدد كيف يجب أن يتغير `gamma` و `beta` في المستقبل (`dgamma`, `dbeta`)
2. يُمرر الخطأ للخلف: يحسب جزء الخطأ الذي يجب أن يمرره للطبقة السابقة (`dx`) حتى تتعلم هي الأخرى

ملاحظة:

- `running_mean` و `running_var` يمكن القول أنهم مثل ذاكرة تراكمية لكل المتوسطات والتباينات التي مرت عليهم الشبكة بكل الدفعات (`batches`) من بداية التدريب للنهاية
- لكن هناك ملاحظة:
- هذه ليست مجرد مجموع بسيط، وإنما هي (Weighted Moving Average)
- ماذا يعني هذا؟ يعني أنها لا تعطي نفس الأهمية لكل الدفعات هي تعطي أهمية أكبر للدفعات الأخيرة وأهمية أقل للدفعات القديمة جداً
- إذا:
- أثناء التدريب: هي ذاكرة قيد التشغيل يتم تحديثها باستمرار مع كل دفعة جديدة
- في نهاية التدريب (وعند الاختبار): تصبح هذه الذاكرة الإحصائيات النهائية والمستقرة التي تمثل مجموعة البيانات التدريبية بأكملها. عندما تريد اختبار الشبكة على صورة جديدة، نستخدم هذه الذاكرة المكتملة لتنظيم البيانات، لأنه لا معنى لحساب متوسط من صورة واحدة

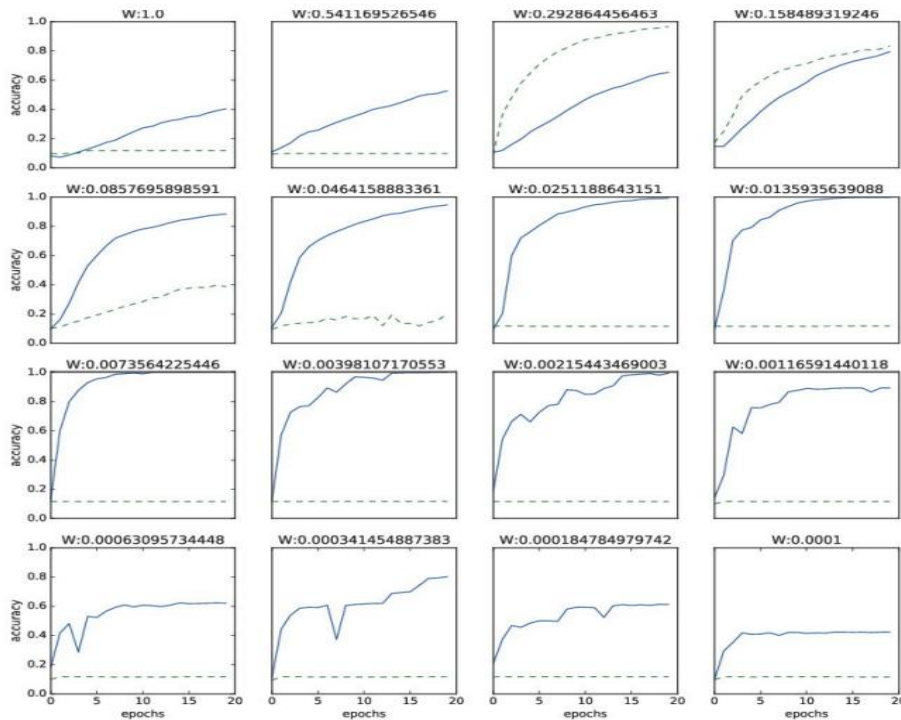
Regularization و Overfitting

الهدف من أي نموذج تعلم آلي ليس أن يكون ببغاء يكرر ما سمعه (بيانات التدريب)، بل أن يفهم المبادئ وامكانيته على استكشاف حالات جديدة لم يرها من قبل (بيانات الاختبار)

لماذا تظهر مشكلة الإفراط في التخصيص (Overfitting): علماً ان BatchNormalization قللت من هي المشكلة

لدينا سببان رئيسيان:

1. النموذج معقد جداً: (Model has many parameters) كأنك تعطي الطالب كتاباً ضخماً جداً وتطلب منه حفظه كلمة بكلمة مثل استخدام شبكة عصبية من 7 طبقات وكل طبقة تحتوي على 100 عقدة هذا يزيد من تعقيد الشبكة
 2. بيانات التدريب قليلة جداً: (Training data is insufficient) تخيل أنك تريد أن تفهم ثقافة دولة بأكملها، لكنك تحدثت فقط مع شخصين ستكون استنتاجاتك مضللة جداً وغير دقيقة مثل استخدام 300 صورة فقط من أصل 60,000 لتدريب نموذج معقد
- كيف منعرف انو عنا Overfitting عن طريق الدقة ؟ حيث استخدمنا ReLU كدالة تنشيط
- لنرى المخططات التالية:



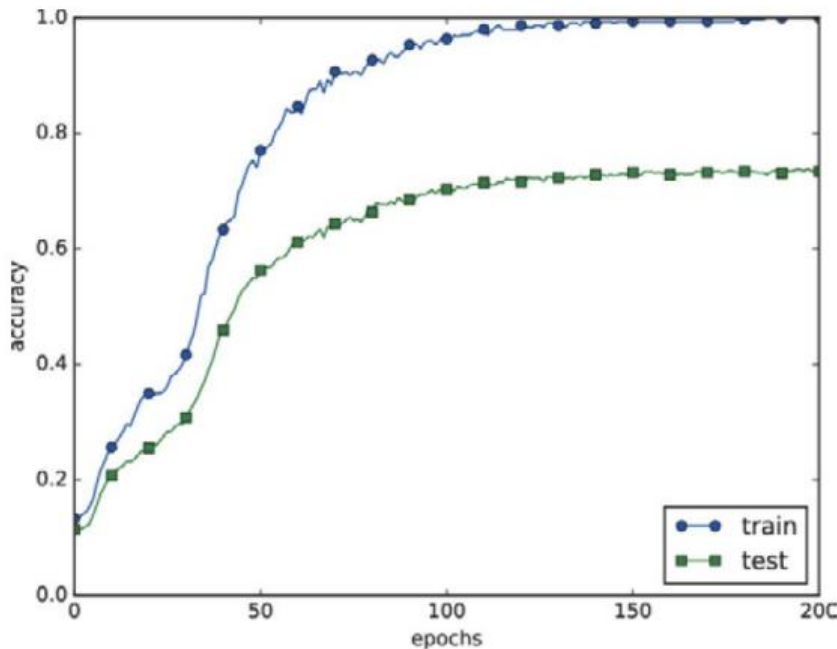
الشرح :

- الخطوط المتصلة (Solid Lines): تمثل أداء الشبكة عندما استخدمنا تقنية Batch Norm ستلاحظ أن دقتها ترتفع بسرعة كبيرة وتصل لمستويات عالية (تعلم سريع ومستقر).

- **الخطوط المنقططة (Dotted Lines):** تمثل أداء الشبكة بدون Batch Norm ستلاحظ أنها أبطأ بكثير في التعلم، وفي بعض الحالات قد لا تتعلم أبداً إذا كانت القيم الأولية للأوزان سيئة
- **دقة التعرف على بيانات التدريب (Training Data):** وصلت لـ 100% قد تظن أن هذا أمر ممتاز، لكن الحقيقة هي العكس! التدريب 100% يعني أن النموذج حفظ الصور الـ 300 التي أعطيتها إياها بصم (مثل طالب حفظ الأسئلة).
- **بيانات الاختبار (Test Data):** هي صور لم يرها النموذج من قبل فالنتيجة كانت أقل بكثير من 100%.
- هذه الفجوة الكبيرة بين أداء النموذج في التدريب وأدائه في الاختبار هي التعريف لـ **Overfitting** التعلم الزائد

ملاحظة:

Batch Norm لوحدها لا تحل مشكلة الـ Overfitting إذا كانت البيانات قليلة والنموذج ضخم لذلك نحن بحاجة لتقنيات أخرى مثل Dropout أو زيادة البيانات لنرى الآن كيف النموذج لا يمكنه التعامل بشكل صحيح مع البيانات العامة (بيانات الاختبار) التي لم تستخدم في التدريب



فالحل هو : التنظيم (Regularization)

التنظيم ليس شيئاً واحداً، بل هو مجموعة من التقنيات التي تضع قيوداً على النموذج أثناء التدريب لمنعه من أن يصبح حافظاً وتجبره على أن يكون متعلماً

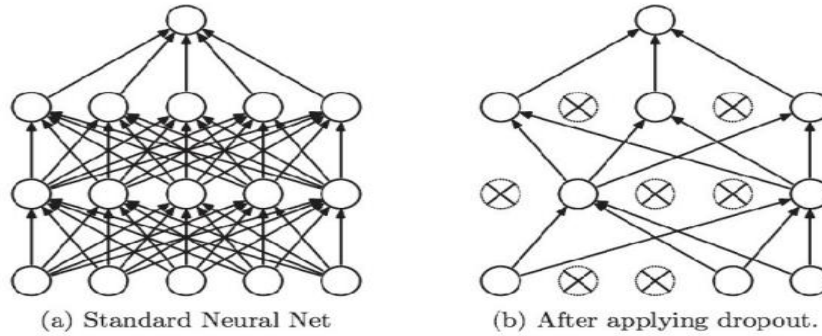
Dropout

هو عملية تعطيل عشوائي ومؤقت للخلايا أثناء التدريب، لجعل الشبكة أكثر مرونة وقدرة على فهم البيانات الجديدة في الواقع، بدلاً من مجرد حفظ بيانات التدريب

كيف يعمل ؟

- **أثناء التدريب (Training):** في كل مرة تمر فيها البيانات، نختار خلايا عصبية (Neurons) عشوائياً ونقوم بإطفائها (نعتبر قيمتها صفراً) هذه الخلايا لا ترسل أي إشارات للطبقة التي بعدها

- تغيير مستمر: في كل دورة تدريبية نطفئ خلايا مختلفة هذا يمنع الخلايا من التآمر مع بعضها لحفظ البيانات بصم (Overfitting)
 - أثناء الاختبار (Testing): هنا لا نطفئ أي خلية نستخدم الشبكة كاملة ، لكننا نضرب المخرجات بنسبة معينة (نفس نسبة الخلايا التي كنا نطفئها) لكي نوازن القوة التي تدرت عليها الشبكة
- مثال :



الشبكة التي على يسار الصورة هي شبكة عصبية عادية ولكن معقدة، لكن التي على يمين الصورة شبكة طبقنا (Dropout) عليها حيث تم اختيار الخلايا العصبية عشوائياً وحذفها لإيقاف نقل الإشارات اللاحقة

الآن لنرى تنفيذ Dropout:

```

61 class Dropout:
62     def __init__(self, dropout_ratio=0.5):
63         self.dropout_ratio = dropout_ratio
64         self.mask = None
65
66     def forward(self, x, train_flg=True):
67         if train_flg:
68             self.mask = np.random.rand(*x.shape) > self.dropout_ratio
69             return x * self.mask
70         else:
71             return x * (1.0 - self.dropout_ratio)
72
73     def backward(self, dout):
74         return dout * self.mask

```

في (forward) :

- أثناء التدريب (train_flg=True) : سوف نرمي نرد عشوائي لكل خلية إذا كان الرقم أصغر من نسبة الـ Dropout ، سوف نضع في mask False (تعطيل)، وإذا كان أكبر نضع True (تشغيل)
- $x * self.mask$: الخلية التي mask لديها يساوي False تصبح صفر وكأنها غير موجودة
- أثناء الاختبار (else) : لا نعطل أي خلية، فقط نضرب المدخلات بـ $(1 - ratio)$ لماذا؟
- لكي نوازن القوة، لأن الشبكة تدرت وهي نصف ميتة، فإذا اشتغلت بكامل طاقتها فجأة ستعطي نتائج ضخمة جداً، فنقوم بتهديتها

في (backward) :

- الخلية التي كانت معطلة في المرور الامامي، تبقى معطلة في المرور الخلفي
- $dout * self.mask$: إذا كان mask صفر هذا يعني لن يمر ولا مشتق (Gradient) من هذه الخلية هذا مثل عمل دالة ReLU

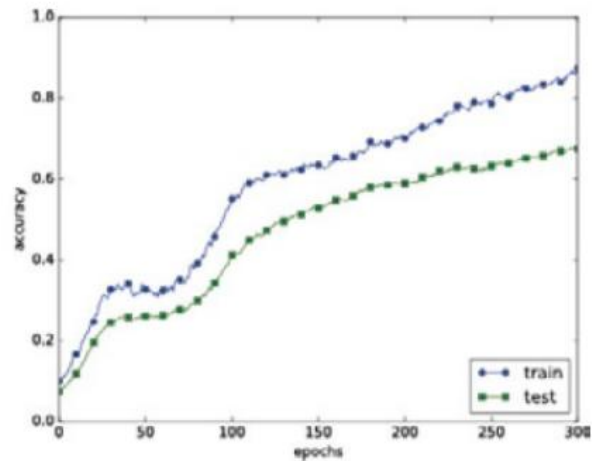
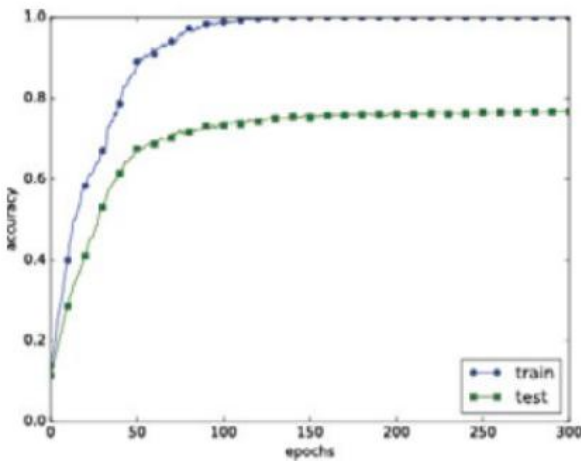
يمكن ان نستنتج: مافائدة الـ Dropout ؟

عندما استخدمنا شبكة من 7 طبقات وبيانات قليلة (300 صورة):

1. قبل الـ Dropout : الدقة في التدريب وصلت 100% (حفظ بصم)، وفي الاختبار كانت سيئة جداً
2. بعد الـ Dropout :

- دقة التدريب انخفضت (لم تصل لـ 100%). وهذا جيد لأنه يعني أن الشبكة لم تعد تحفظ
- الفجوة بين التدريب والاختبار صغرت جداً
- الشبكة أصبحت قادرة على فهم الصور الجديدة (التعميم)

لنرى ذلك :

ما هو الـ Ensemble Learning (التعلم الجماعي)؟

تخيل أنك تريد تشخيص مرض معين بدلاً من سؤال طبيب واحد، قمت بسؤال 5 أطباء مختلفين، ثم أخذت متوسط آرائهم.

- في الشبكات العصبية: نقوم بتدريب 5 شبكات (مثلاً) بشكل منفصل
- وقت الاختبار: نعطي الصورة لـ 5 شبكات، ونأخذ متوسط النتائج التي خرجت منها
- النتيجة: أثبتت التجارب أن هذا الأسلوب يرفع الدقة بنسبة كبيرة لأن الأخطاء الفردية لكل شبكة تلغي بعضها البعض

كيف يقلد الـ Dropout الـ Ensemble Learning ؟

بدلاً من تدريب 5 شبكات مستقلة (علماً أن هذا مكلف ويحتاج وقت)، نستخدم Dropout في شبكة واحدة:

- أثناء التدريب: في كل مرة تمر فيها البيانات، نقوم بتعطيل خلايا عشوائية هذا يعني أننا في كل مرة ندرب شكل جديد أو موديل مصغر من الشبكة

- التفسير: إذا قمنا بالتدريب 1000 مرة، فكأننا قمنا بتدريب 1000 موديل مختلف (لأن كل مرة كانت الخلايا المعطلة مختلفة)

الربط بينهما (أثناء التنبؤ)

- في الـ Ensemble: نجمع نتائج الـ 5 شبكات ونقسم على 5 (متوسط)
- في الـ Dropout: بما أننا لا نستطيع تشغيل 1000 نسخة، نقوم بتشغيل الشبكة كاملة ونضرب المخرجات في نسبة الإطفاء (مثلاً 0.5) هذا الضرب يعادل تماماً عملية أخذ المتوسط لجميع الموديلات التي تدربت أثناء الـ Dropout

الخلاصة:

الـ Dropout ليس مجرد أداة لحذف الخلايا، بل هو وسيلة اقتصادية لعمل Ensemble Learning (ذكاء جماعي) داخل شبكة واحدة فقط. هذا يجعل الشبكة أقوى بكثير وأقل عرضة للوقوع في فخ الحفظ (Overfitting)، لأنها لم تتعلم ك فرد بل ك مجموعة من الاحتمالات.

Validating Hyperparameters

الفرق بين Parameters و Hyperparameters:

- المعاملات العادية (Parameters): هي التي يتعلمها النموذج بنفسه (مثل الأوزان والانحيازات)
- أما المعاملات الفائقة (Hyperparameters): هي بارامترات (الإعدادات) نحن نختار قيمها قبل بدء التدريب، ولا تتعلمها الشبكة بنفسها

- أمثلة: عدد الطبقات، عدد الخلايا، معدل التعلم (Learning Rate)، حجم الدفعة (Batch Size)، عدد الخلايا العصبية في كل طبقة....
- أهميتها: اختيار قيم خاطئة لهذه المعاملات يؤدي لفشل النموذج تماماً
- سابقا كنا نقسم الداتا التي لدينا لقسمين قسم تدريب وقسم اختبار وكنا بكل مرة ندرب على بيانات التدريب ونختبر على بيانات الاختبار والغرض من بيانات الاختبار كان هو نحسب الدقة وكنا نرى بيانات الاختبار كم الدقة التي تعطيها واختار أعلى قيمة أعطتها لكن إذا استخدمنا بيانات الاختبار حتى نحدد Hyperparameters هذا الشيء أدى لظهور مشكلة ما هي؟
- أصبح و كأننا نتعلم من بيانات الاختبار أي المعاملات الفائقة ستحفظ بيانات الاختبار (Overfit to test data).
- أصبح لدينا حالة تدعى تسرب معلومات لإننا لا نستطيع أن نقرب من بيانات الاختبار هذا يعني لن نعرف أبداً أداء النموذج الحقيقي في الواقع فما هو الحل؟

الحل هو تقسيم البيانات لثلاثة أقسام:

لضمان أن النموذج يفهم ولا يحفظ، نقسم البيانات إلى:

1. بيانات التدريب: (Training Data) لتحديد الأوزان والانحيازات عادة تكون 0.7 ونقسم بيانات الاختبار ل: 0.3
2. بيانات التحقق (Validation Data): مهم جداً تستخدم فقط لاختيار أفضل المعاملات الفائقة Hyperparameters عادة 0.15
- بيانات الاختبار (Test Data): تستخدم مرة واحدة فقط في النهاية لتقييم الأداء النهائي 0.15

كيف نبحث عن أفضل القيم؟ Optimizing Hyperparameters

من خلال البحث العشوائي:

- لماذا البحث العشوائي أفضل من المنظم (Grid Search)؟ لأن بعض المعاملات تؤثر على النتيجة أكثر من غيرها، والبحث العشوائي يغطي مساحات متنوعة بشكل أسرع
- نطاق البحث: نبحث باستخدام قوى الرقم 10 (Log Scale)، مثلاً من 10^{-3} إلى 10^3

كيفية البحث عن افضل Hyperparameters :

1. حدد نطاق واسع: لكل Hyperparameters حدد نطاقاً كبيراً (مثلاً سرعة التعلم من 0.001 إلى 100)
2. اختر بشكل عشوائي: بدلاً من تجربة كل قيمة ممكنة (مما يأخذ وقت طويل)، اختر combinations بشكل عشوائي من هذه النطاقات
3. درّب وقيّم: درّب النموذج باستخدام هذه المعاملات العشوائية على بيانات التدريب، ثم قيّم أدائه على بيانات التحقق
4. كرّر وضيق النطاق: كرر هذه العملية (100 مرة مثلاً) بعد عدة تجارب، ستبدأ في ملاحظة أن بعض القيم تعطي نتائج أفضل عندئذ ضيق نطاق البحث حول هذه القيم الجيدة وكرر العملية
5. تسريع العملية: لتسريع هذه العملية، يمكنك تدريب النموذج لعدد أقل من الحقب (epochs) خلال البحث

ماهي الخيارات الأساسية التي يجب اتخاذها عند بناء وتدريب الشبكة العصبية؟

1. عدد الطبقات المخفية (Number of hidden layers): أي عمق الشبكة
2. عدد الخلايا العصبية في كل طبقة (Number of neurons per layer): أي عرض الشبكة
3. دوال التنشيط (Activation functions):
4. معدل التعلم (Learning rate): المعامل الذي يتحكم في حجم الخطوات التي يتخذها النموذج لتحديث أوزانه أثناء التدريب
5. حجم الدفعة (Batch size): عدد عينات البيانات التي تعالجها الشبكة قبل تحديث أوزانها

6. عدد الحقبة (Number of epochs): يحدد عدد المرات التي سيتم فيها تمرير مجموعة بيانات التدريب بأكملها عبر الشبكة حيث يتحكم في مدة التدريب

الخلاصة:

بما أن التجريب اليدوي لكل هذه الاحتمالات مستحيل، فإن الحل هو البحث العشوائي المنظم باستخدام بيانات التحقق حصراً، مع تقليل وقت كل تجربة (Small Epochs) حتى نصل لأفضل تركيبة لنرى الكود التالي لاختيار أفضل معاملات فائقة للنموذج والذي يمثل Grid search

```
Split data → train, validation, test

best_score = -∞
best_params = None

For each learning_rate in [0.001, 0.01, 0.1]:
    For each hidden_size in [10, 50, 100]:
        For each batch_size in [16, 32, 64]:
            .....
            # 1. Initialize a neural network with that configuration
            model = NeuralNetwork(hidden_size=hidden_size)

            # 2. Train model on training set
            model.train(
                X_train, y_train,
                learning_rate=learning_rate,
                batch_size=batch_size,
                epochs=E
            )

            # 3. Compute accuracy on validation set
            score = model.evaluate(X_valid, y_valid)

            # 4. Update best parameters
            if score > best_score:
                best_score = score
                best_params = {
                    'learning_rate': learning_rate,
                    'hidden_size': hidden_size,
                    'batch_size': batch_size
                }

            # 5. Retrain using train+valid data and best params
            model = NeuralNetwork(hidden_size=best_params["hidden_size"])
            model.train(
                X_trainval, y_trainval,
                learning_rate=best_params["learning_rate"],
                batch_size=best_params["batch_size"],
                epochs=E
            )

            # 6. Evaluate on test set
            test_acc = model.evaluate(X_test, y_test)
            Print best params, best validation score, and test score
```

Gener

- أول خطوة هي تقسيم البيانات: يتم تقسيم مجموعة البيانات الكاملة إلى ثلاث مجموعات منفصلة تماماً:
- 1. التدريب: (Training) تُستخدم حصراً لتدريب النماذج
- 2. التحقق: (Validation) تُستخدم حصراً لمقارنة أداء النماذج المختلفة واختيار أفضل معاملات فائقة
- 3. الاختبار: (Test) تُستخدم مرة واحدة فقط في النهاية لتقييم الأداء النهائي للنموذج المختار

■ تهيئة كل من :

best_score = $-\infty$:

يتم تعيين أفضل درجة إلى سالب ما لا نهاية له لضمان أن أول نموذج يتم تقييمه سيحل محل هذه القيمة

best_params = None :

يتم تهيئة متغير فارغ ليحتفظ بقيم المعاملات الفائقة التي حققت best_score

■ عملية البحث والتقييم :

- الحلقات (Loops): الحلقات المتداخلة تقوم بإنشاء كل تركيبة ممكنة من قيم المعاملات الفائقة المحددة هذه عملية بحث شامل (Grid Search) لاستكشاف مساحة البحث
- داخل كل حلقة (لكل تركيبة):

1. إنشاء وتدريب النموذج: يتم إنشاء نسخة جديدة من النموذج بالمعاملات الفائقة الحالية يتم تدريب هذا النموذج فقط على بيانات التدريب
2. التقييم على بيانات التحقق: بعد انتهاء التدريب، يتم قياس أداء النموذج باستخدام بيانات التحقق للحصول على درجة (score)

3. تحديث أفضل إعدادات: تتم مقارنة الدرجة التي تم الحصول عليها مع best_score المسجلة إذا كانت الدرجة الجديدة أعلى، فهذا يعني أن هذه التركيبة من المعاملات هي الأفضل حتى الآن في هذه الحالة يتم تحديث best_score بالقيمة الجديدة وتُخزن قيم المعاملات الحالية في best_params

■ التدريب النهائي والتقييم النهائي:

بعد ما حددنا أفضل المعاملات الفائقة

1. إعادة التدريب باستخدام أفضل المعاملات: يتم إنشاء نموذج نهائي جديد هذا النموذج يتم تدريبه باستخدام أفضل المعاملات الفائقة التي تم العثور عليها (best_params) بشكل مهم، يتم تدريبه على مجموعة بيانات أكبر هي اتحاد بيانات التدريب والتحقق (train + valid) للاستفادة من كل البيانات المتاحة (ما عدا بيانات الاختبار).
2. التقييم النهائي على بيانات الاختبار: يتم تقييم النموذج النهائي المدرب مرة واحدة فقط على بيانات الاختبار. هذه الخطوة التي توفر تقييماً نهائياً وغير متحيز لأداء النموذج على بيانات لم يرها أبداً خلال أي من مراحل التدريب أو اختيار المعاملات.

النهاية